

TechTribers

Java Interview Series · Part 11 · Q428–Q479

Object-Oriented Programming

The four pillars, SOLID, composition vs inheritance, constructors and initialisation order, binding and overloading, enums, design techniques and eight low-level design problems, every example run on a real JDK.

52	51	24	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `OP-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

52	6	51	24
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🧱	01	OOP Foundations & Design Thinking	Q428–Q441	5
🏗️	02	SOLID Principles	Q442–Q447	20
⚙️	03	Inheritance & Polymorphism: How It Really Works	Q448–Q461	26
🧿	04	Enums as Objects	Q462–Q464	41
🧠	05	Object Modelling Techniques	Q465–Q471	44
🧩	06	Low-Level Design: Object-Oriented Problems	Q472–Q479	51

© TechTribers. For personal use only. Do not redistribute.

All questions

🧱 OOP Foundations & Design Thinking

Q428 What are the four pillars of OOP, and how do they work together in real code? 5

Q429 What is the difference between abstraction and encapsulation? 6

Q430 Is Java a purely object-oriented language? 7

Q431 What is the difference between procedural and object-oriented programming? 8

Q432 Is Java pass-by-value or pass-by-reference? 9

Q433 What are coupling and cohesion? 10

Q434 What is the difference between association, aggregation and composition? 11

Q435 What does "favour composition over inheritance" mean, and when is inheritance still right? 12

Q436 What is the fragile base class problem? 14

Q437 What is the Law of Demeter ("don't talk to strangers")? 15

Q438 What is "Tell, Don't Ask", and why is an anemic domain model an anti-pattern? 16

Q439 What are DRY, KISS and YAGNI, and when do they conflict? 17

Q440 How does polymorphism replace if/else and switch chains? 18

Q441 What is dependency injection, and how is it different from inversion of control? 19

🏗️ SOLID Principles

Q442 What are the SOLID principles? Give a one-line explanation of each. 20

Q443 Explain the Single Responsibility Principle with an example. 21

Q444 Explain the Open/Closed Principle with an example. 22

Q445 Explain the Liskov Substitution Principle with an example. 22

Q446 Explain the Interface Segregation Principle with an example. 24

Q447 Explain the Dependency Inversion Principle with an example. 25

⚙️ Inheritance & Polymorphism: How It Really Works

Q448 What types of inheritance does Java support? 26

Q449 Are constructors inherited? Can they be overridden? 27

Q450 In what order are static blocks, instance blocks and constructors executed in an inheritance chain? 28

Q451 What happens if a constructor calls an overridable method? 29

Q452 What is the difference between static (compile-time) and dynamic (runtime) binding? 30

Q453 How does the compiler choose between overloaded methods? 31

Q454 Are fields polymorphic? What is field hiding? 32

Q455 What are the rules for overriding a method (access, return type, exceptions)? 33

Q456 What are upcasting and downcasting, and when does ClassCastException occur? 34

Q457 Can a subclass in another package access a protected member through a parent reference? 35

Q458 How does the JVM implement method calls and dynamic dispatch? 36

Q459 How do equals() and hashCode() behave in an inheritance hierarchy? 37

Q460 What methods does the java.lang.Object class provide? 38

Q461 What is a marker interface, and how does it compare with an annotation? 39

🧿 Enums as Objects

Q462 Are enums just constants? What can an enum contain in Java? 41

Q463 How do you give each enum constant its own behaviour? 42

Q464 Why use EnumMap and EnumSet instead of HashMap and HashSet? 43

© TechTribers. For personal use only. Do not redistribute.

🧠 Object Modelling Techniques

Q465 What does "program to an interface, not an implementation" mean? 44

Q466 What is the difference between a value object and an entity? 45

Q467 What are static factory methods, and why prefer them to constructors? 46

Q468 Why are Java arrays covariant but generics invariant? 47

Q469 What is the Visitor pattern, and what is double dispatch? 47

Q470 What is the Null Object pattern? 49

Q471 How do you design a class for inheritance, and when should a class be final? 50

🧩 Low-Level Design: Object-Oriented Problems

Q472 Design a parking lot. 51

Q473 Design a vending machine using the State pattern. 53

Q474 Design an ATM (cash dispensing and transaction flow). 54

Q475 Design a library management system. 56

Q476 Design a tic-tac-toe game that scales to n × n. 58

Q477 Design an elevator system. 59

Q478 Design a shopping cart with flexible pricing rules (discounts, coupons, tax). 61

Q479 Design a notification system (email, SMS, push) with topics and subscribers. 62

© TechTribers. For personal use only. Do not redistribute.

Section 01

OOP Foundations & Design Thinking

The ideas behind object-oriented design: how the pillars fit together, how objects relate, and the principles that keep code flexible.

Questions 428–441 · 14 questions



Q428 OP-001 • Basic

What are the four pillars of OOP, and how do they work together in real code?

⚡ Quick answer

Abstraction exposes what an object does and hides how. **Encapsulation** keeps state private behind methods that enforce rules. **Inheritance** lets a subtype reuse and extend a parent. **Polymorphism** lets one call behave differently depending on the object. Real designs use all four at once.

Interview tip: don't recite four definitions. Point at one example and show where each pillar is visible. Here `Account` is the abstraction, `balance` is hidden state, `Savings` / `Current` reuse the parent, and the loop never asks which kind of account it has.

Modern Java adds two more tools in the same spirit: **interfaces with default methods** (behaviour reuse without a class hierarchy) and **sealed types + records** (closed, immutable data models).

Java

```
abstract class Account {                                    // ABSTRACTION: callers see what an
account does
    private double balance;                                 // ENCAPSULATION: state is private
    protected Account(double opening) { balance = opening; }
    public final double balance() { return balance; }
    public void deposit(double amt) { if (amt <= 0) throw new IllegalArgumentException("amount");
balance += amt; }
    public void withdraw(double amt) {
        if (amt > balance + overdraftLimit()) throw new IllegalStateException("insufficient funds");
        balance -= amt;
    }
    protected abstract double overdraftLimit();             // hook: subclasses decide the rule
    public abstract String type();
}
class Savings extends Account {                             // INHERITANCE: reuse the shared
behaviour
    Savings(double o) { super(o); }
    protected double overdraftLimit() { return 0; }
    public String type() { return "Savings"; }
}
class Current extends Account {
```


© TechTribers. For personal use only. Do not redistribute.

```
    Current(double o) { super(o); }
    protected double overdraftLimit() { return 1000; }
    public String type() { return "Current"; }
}
List<Account> accounts = List.of(new Savings(500), new Current(500));
for (Account a : accounts) {                          // POLYMORPHISM: same call, different
behaviour
    try { a.withdraw(1200); System.out.println(a.type() + " ok, balance=" + a.balance()); }
    catch (IllegalStateException e) { System.out.println(a.type() + " refused: " + e.getMessage());
}
}
```

▶ Output

```
Savings refused: insufficient funds
Current ok, balance=-700.0
```

Abstraction	Encapsulation	Inheritance	Polymorphism
show WHAT, hide HOW	protect state behind methods	reuse and specialise	one call, many behaviours
`interface` · `abstract class`	`private fields` · validation	`extends` · `super`	`overriding` · `interfaces`

Together: callers use an abstraction (interface), the object protects its own state (encapsulation), shared code lives in a parent (inheritance), and each subtype supplies its own behaviour (polymorphism).

⚠️ **Interview trap**

Inheritance is the pillar most often overused. If the only reason for `extends` is code reuse and the relationship isn't truly "is-a", use composition (see the composition vs inheritance question).

🔍 **Q429** OP-002 • Basic

What is the difference between abstraction and encapsulation?

⚡ **Quick answer**

Abstraction is a design-level idea: expose a simple contract (the *what*) and hide complexity. **Encapsulation** is an implementation-level mechanism: bundle data with the methods that guard it and restrict direct access. Abstraction decides what to show; encapsulation enforces what stays hidden.

	Abstraction	Encapsulation
Level	Design (what to expose)	Implementation (how to protect)
Question answered	"What can I do with this?"	"Who may touch this state?"
Java tools	`interface` , `abstract class`	`private` , getters with validation, immutability

© TechTribers. For personal use only. Do not redistribute.

Benefit	Swap implementations, lower coupling	Protect invariants, safe refactoring

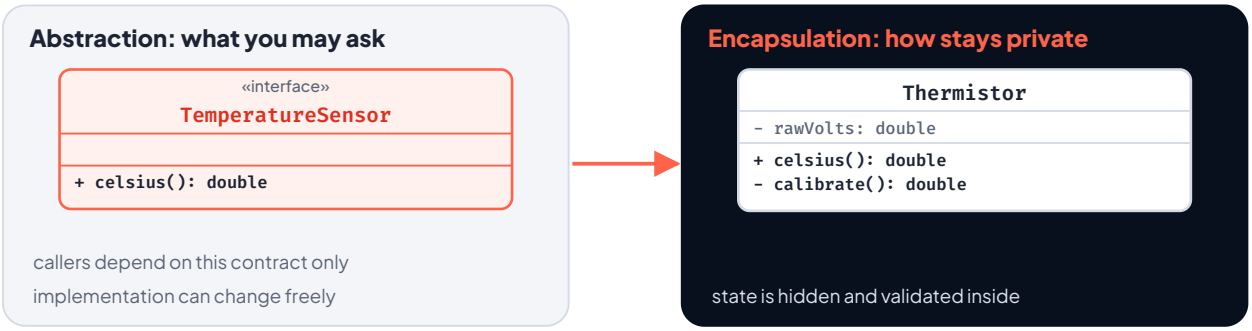
Java

```java
interface TemperatureSensor { double celsius(); }                    // ABSTRACTION: the contract callers use

final class Thermistor implements TemperatureSensor {
    private final double rawVolts;                                   // ENCAPSULATION: raw data stays private
    Thermistor(double rawVolts) { this.rawVolts = rawVolts; }
    public double celsius() { return Math.round((rawVolts * 100 - 50) * 10) / 10.0; }   // conversion hidden inside
}
TemperatureSensor s = new Thermistor(0.75);
System.out.println(s.celsius());
```

▶ Output

```
25.0
```



💡 **Pro tip**

A one-line answer interviewers like: "Abstraction hides complexity by showing only essentials; encapsulation hides data by controlling access to it. Abstraction is achieved through encapsulation."

☕ **Q430** OP-003 • Basic

Is Java a purely object-oriented language?

⚡ **Quick answer**

No. Java is *mostly* object-oriented, but it has eight **primitive types** that aren't objects, `static` members that belong to classes rather than instances, and `main` itself is a static method. Wrapper classes (`Integer` , `Double`) and autoboxing bridge the gap.

Why primitives exist: speed and memory. An `int` is 4 bytes inline, while an `Integer` is a heap object plus a reference. Project Valhalla (value classes, in preview from JDK 28) is the long-term plan to give object-like types

© TechTribers. For personal use only. Do not redistribute.

primitive-like performance and blur this line.

```java
int a = 5;                          // primitive: not an object, no methods, no null
Integer boxed = a;                  // wrapper object created by autoboxing
System.out.println(boxed.getClass().getSimpleName() + " " + boxed.equals(5));
System.out.println(Math.max(2, 3)); // static: called on the class, no object involved
int[] arr = {1, 2};                 // arrays ARE objects
System.out.println(arr.getClass().getSimpleName() + " " + (arr instanceof Object));
```

▶ Output

```
Integer true
3
int[] true
```

⚠️ Interview trap

Pure OO languages (Smalltalk, Ruby) treat everything, including numbers, as an object. Saying "Java is fully OO because everything is a class" is wrong; every statement must live in a class, but not every value is an object.

Q431 OP-004 • Basic

What is the difference between procedural and object-oriented programming?

⚡ Quick answer

Procedural code separates **data** from the **functions** that act on it and organises programs as a sequence of procedures. OOP groups data and behaviour into **objects**, so a type owns its operations. OOP makes adding new *types* easy; procedural code makes adding new *operations* easy.

	Procedural (C, Pascal)	Object-oriented (Java, C#)
Unit of design	Function	Class / object
Data	Global or passed around, exposed	Hidden inside objects
Reuse	Copy / call functions	Inheritance, composition, interfaces
Easy to add	New operations	New types
Hard to add	New data types (edit every function)	New operations (edit every class)

```java
// Procedural style: one function must know every shape
static double area(String type, double... d) {
    return switch (type) {
        case "circle" -> Math.PI * d[0] * d[0];
        case "rect" -> d[0] * d[1];
        default -> throw new IllegalArgumentException(type);
```


© TechTribers. For personal use only. Do not redistribute.

```
    };
}
// OO style: each shape owns its behaviour; a new shape touches no existing code
interface Shape { double area(); }
record Circle(double r) implements Shape { public double area() { return Math.PI * r * r; } }
record Rect(double w, double h) implements Shape { public double area() { return w * h; } }

System.out.printf("%.2f %.2f%n", area("circle", 2), new Circle(2).area());
System.out.println(new Rect(3, 4).area());
```

▶ Output

```
12.57 12.57
12.0
```

💡 **Pro tip**

The last two rows are the *expression problem*. Modern Java's sealed types and pattern-matching `switch` give OO code the procedural strength too: you can add operations without touching the classes.

Q432 OP-005 • Intermediate

Is Java pass-by-value or pass-by-reference?

⚡ **Quick answer**

Java is **always pass-by-value**. For objects, the value that is copied is the *reference*. So a method can change the object it points to, but reassigning its parameter never changes the caller's variable.

Mental model: a variable of an object type holds a *remote control*, not the TV. Passing it to a method photocopies the remote. Both remotes control the same TV (mutation is visible), but pointing your copy at a new TV does nothing to the original.

This is why `swap(a, b)` can't be written for two objects or two ints in Java: the method only receives copies of the references.

Java

```
static void reassign(StringBuilder sb) { sb = new StringBuilder("new"); }   // changes only the local copy of the reference
static void mutate(StringBuilder sb) { sb.append("!"); }                    // follows the copy to the SAME object
static void bump(int x) { x++; }                                            // primitive copy

StringBuilder s = new StringBuilder("hi");
reassign(s);
System.out.println(s);       // hi   : the caller's reference is untouched
mutate(s);
System.out.println(s);       // hi!  : both references pointed at the same object
int n = 1;
bump(n);
System.out.println(n);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
hi
hi!
1
```

⚠️ Interview trap

"Objects are passed by reference" is the classic wrong answer. True pass-by-reference (as with C++ `&`) would let `reassign` change `s`. Java never does.

Q433 OP-006 • Intermediate

What are coupling and cohesion?

⚡ Quick answer

Cohesion measures how closely the responsibilities inside one class belong together (aim high). **Coupling** measures how much one class depends on others' details (aim low). Good design means focused classes that talk to each other through small, stable abstractions.

Signs of **low cohesion**: a class named `Manager` / `Util` with unrelated methods, methods that use different subsets of the fields, or a description that needs the word "and".

Signs of **high coupling**: changing one class forces edits in many others, `new` of concrete classes everywhere, long chains like `a.getB().getC().doIt()`, and tests that need half the system to run.

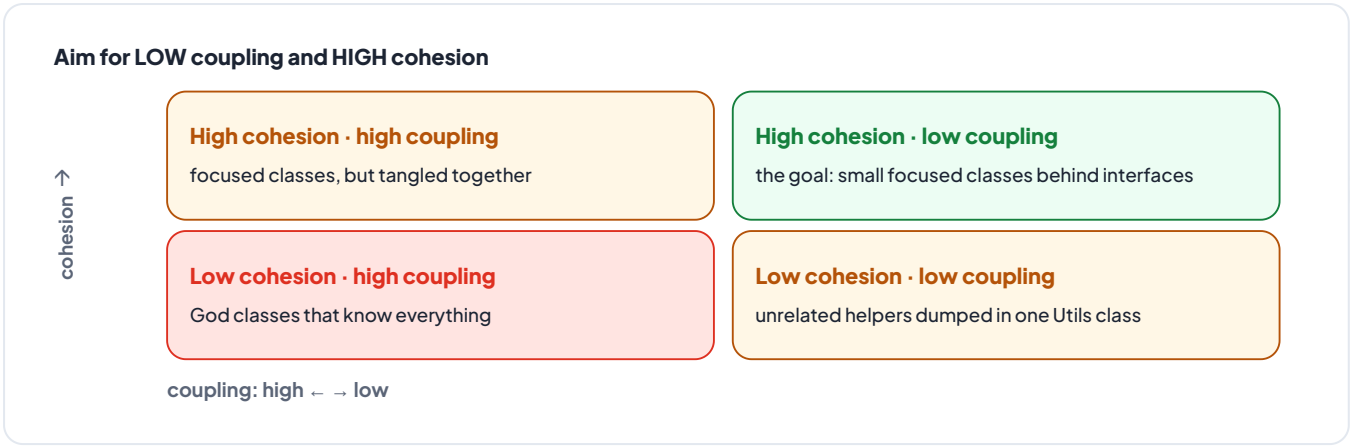
Java

```java
// Tight coupling: the service creates and knows its concrete dependency
class TightOrderService {
    private final EmailSender sender = new EmailSender();
    void place(String id) { sender.send("Order " + id + " placed"); }
}
// Loose coupling: depends on an abstraction supplied from outside
interface Notifier { void send(String msg); }
class EmailSender implements Notifier { public void send(String m) { System.out.println("EMAIL: " +
m); } }
class SmsSender implements Notifier { public void send(String m) { System.out.println("SMS: " + m);
} }
class OrderService {
    private final Notifier notifier;
    OrderService(Notifier n) { notifier = n; }
    void place(String id) { notifier.send("Order " + id + " placed"); }
}
new OrderService(new EmailSender()).place("A1");
new OrderService(new SmsSender()).place("A2");
```

▶ Output

```
EMAIL: Order A1 placed
SMS: Order A2 placed
```


© TechTribers. For personal use only. Do not redistribute.



💡 **Pro tip**

Practical heuristic: if you can't describe a class in one short sentence without "and", split it. If you can't test a class without starting its dependencies, introduce an interface.

Q434 OP-007 • Intermediate

What is the difference between association, aggregation and composition?

⚡ **Quick answer**

All three describe "has-a" relationships with increasing ownership. **Association**: one object uses another. **Aggregation**: a whole holds parts that can exist independently. **Composition**: a whole owns parts whose lifetime is bound to it.

The practical test is **lifecycle and sharing**: if deleting the whole should delete the part and the part can't be shared, it is composition. Otherwise it is aggregation or plain association. In UML, composition is a filled diamond, aggregation a hollow diamond, association a plain arrow.

In Java the distinction is by design intent, not syntax. Typical hints: composition creates the part inside (`new Engine()`) or in the constructor; aggregation receives parts from outside and keeps references.

Java

```
class Engine { final int hp; Engine(int hp) { this.hp = hp; } }
class Car {                                        // COMPOSITION: Car creates and owns its Engine
    private final Engine engine = new Engine(150);
    int hp() { return engine.hp; }
}
class Driver { final String name; Driver(String n) { name = n; } }   // exists independently
class Trip {                                       // ASSOCIATION: Trip just uses a Driver
    final Driver driver; Trip(Driver d) { driver = d; }
}
class Fleet {                                      // AGGREGATION: Fleet holds Cars that live on
their own
    private final List<Car> cars = new ArrayList<>();
    void add(Car c) { cars.add(c); }
    int size() { return cars.size(); }
```

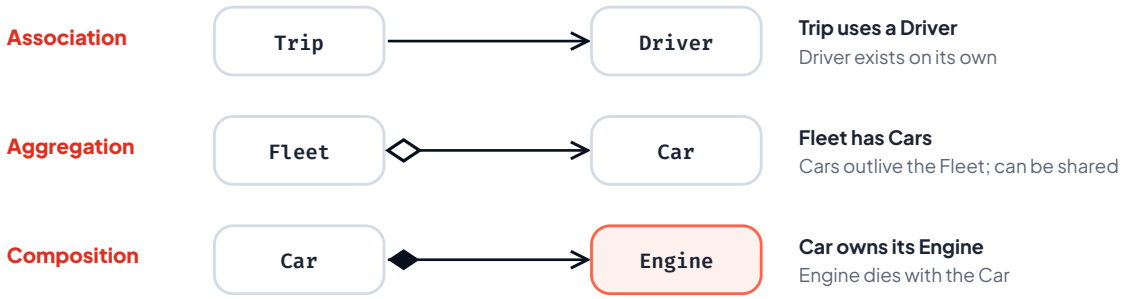

© TechTribers. For personal use only. Do not redistribute.

```
}
Car shared = new Car();
Fleet f1 = new Fleet(), f2 = new Fleet();
f1.add(shared); f2.add(shared);                    // the same Car belongs to two fleets:
aggregation allows sharing
System.out.println(f1.size() + " " + f2.size() + " engine hp=" + shared.hp());
System.out.println(new Trip(new Driver("Asha")).driver.name);
```

▶ Output

```
1 1 engine hp=150
Asha
```

Association · Aggregation · Composition



⚠ Interview trap

Interviewers often ask for an example of composition in the JDK: `ArrayList` owns its internal array; a `Thread` doesn't own the `Runnable` it receives (aggregation or association).

Q435 OP-008 • Intermediate

What does "favour composition over inheritance" mean, and when is inheritance still right?

⚡ Quick answer

Prefer building behaviour by **combining objects** (has-a) rather than extending classes (is-a). Composition is more flexible (behaviour can change at runtime), avoids deep hierarchies, and doesn't expose a parent's internals. Use inheritance only for a genuine, stable "is-a" relationship that respects the parent's contract.

With inheritance, "flying quacking duck", "non-flying quacking duck" and "flying silent duck" each need a subclass, and the number of subclasses grows with every new trait. With composition, each trait is a small interface and a duck picks implementations.

Inheritance is still right when: the subtype truly *is a* parent in every context (LSP holds), the parent was designed for extension (documented hooks, template methods), and you control both classes. Examples: `ArrayList extends AbstractList`, framework base classes.

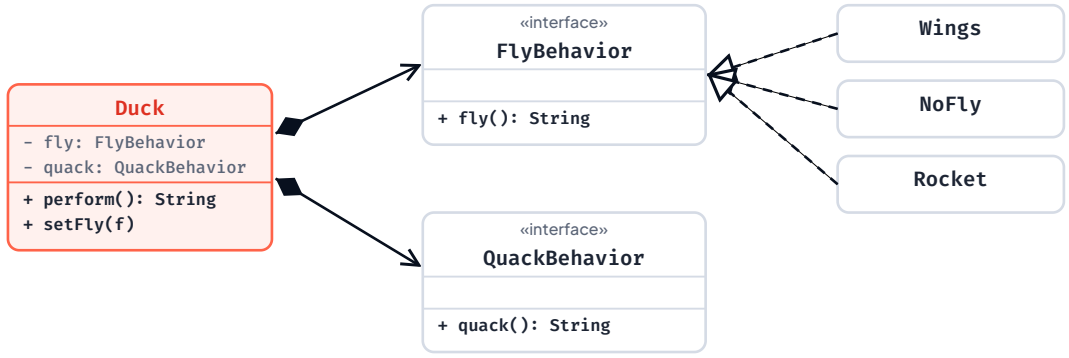
© TechTribers. For personal use only. Do not redistribute.

Java

```java
interface FlyBehavior { String fly(); }
interface QuackBehavior { String quack(); }
class Duck {
    private FlyBehavior fly;                                    // behaviours are parts, not
subclasses
    private final QuackBehavior quack;
    Duck(FlyBehavior f, QuackBehavior q) { fly = f; quack = q; }
    void setFly(FlyBehavior f) { fly = f; }                     // change behaviour at runtime
    String perform() { return fly.fly() + " & " + quack.quack(); }
}
FlyBehavior wings = () -> "flies";
FlyBehavior grounded = () -> "can't fly";
QuackBehavior loud = () -> "QUACK";
Duck d = new Duck(wings, loud);
System.out.println(d.perform());
d.setFly(grounded);                                             // e.g. the duck is injured
System.out.println(d.perform());
```

▶ Output

```
flies & QUACK
can't fly & QUACK
```



Behaviours are parts you can swap at runtime (setFly), instead of a subclass for every combination.

💡 **Pro tip**

Delegation plus an interface is the usual replacement: hold a reference to the parent type, forward the calls you need, and keep control over which behaviour is exposed.

© TechTribers. For personal use only. Do not redistribute.

Q436 OP-009

• Advanced

What is the fragile base class problem?

⚡ Quick answer

A subclass depends on **how** its parent is implemented, not just on its contract. When the parent changes internals (for example, one method starts calling another), the subclass can break even though the parent's public API is unchanged. Inheritance breaks encapsulation.

This is Joshua Bloch's example from *Effective Java* (Item 18). `AbstractCollection.addAll` is implemented by calling `add` for each element: an implementation detail the subclass accidentally depended on. If a later JDK changed that, the counter would silently change again.

Java

```
class InstrumentedSet<E> extends HashSet<E> {                    // count how many elements were
added
    private int addCount = 0;
    @Override public boolean add(E e) { addCount++; return super.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) { addCount += c.size(); return
super.addAll(c); }
    int addCount() { return addCount; }
}
var broken = new InstrumentedSet<String>();
broken.addAll(List.of("a", "b", "c"));
System.out.println("expected 3, got " + broken.addCount());   // HashSet.addAll() calls add()
internally: counted twice

// Fix: composition + delegation. We own the counting logic and never rely on the parent's self-use
class CountingSet<E> {
    private final Set<E> inner = new HashSet<>();
    private int count;
    boolean add(E e) { count++; return inner.add(e); }
    boolean addAll(Collection<? extends E> c) { boolean changed = false; for (E e : c) changed |=
add(e); return changed; }
    int count() { return count; }
}
var fixed = new CountingSet<String>();
fixed.addAll(List.of("a", "b", "c"));
System.out.println("count = " + fixed.count());
```

▶ Output

```
expected 3, got 6
count = 3
```

⚠️ Interview trap

"Just don't override addAll" misses the point: you can't know which overridable methods call which without reading the parent's source, and that knowledge can go stale. Design for inheritance deliberately or forbid it with `final`.

© TechTribers. For personal use only. Do not redistribute.

Q437 OP-010

• Intermediate

What is the Law of Demeter ("don't talk to strangers")?

⚡ Quick answer

A method should only call methods on: itself, its parameters, objects it creates, and its direct fields. Chains like `customer.getWallet().getMoney()` make the caller depend on the internal structure of several classes. Ask the nearest object to do the work instead.

Benefit: `Customer` can change how it pays (card, UPI, wallet) without breaking `Shop`. Cost: more small delegating methods. Fluent builders and stream pipelines (`list.stream().map(...).filter(...)`) look like chains but are fine: each call returns the *same kind* of object, not a deeper internal one.

Java

```
class Wallet {
    private double money = 100;
    double money() { return money; }
    void debit(double a) { money -= a; }
}
class Customer {
    private final Wallet wallet = new Wallet();
    Wallet getWallet() { return wallet; }
    boolean pay(double a) { if (wallet.money() < a) return false; wallet.debit(a); return true; }
// behaviour lives here
}
class Shop {
    // Violates Demeter: Shop knows Customer has a Wallet with money() and debit()
    boolean badCheckout(Customer c, double price) {
        if (c.getWallet().money() >= price) { c.getWallet().debit(price); return true; }
        return false;
    }
    // Follows Demeter: ask the customer to pay and stay ignorant of wallets
    boolean checkout(Customer c, double price) { return c.pay(price); }
}
Shop shop = new Shop();
System.out.println(shop.checkout(new Customer(), 60) + " " + shop.checkout(new Customer(), 150));
```

▶ Output

```
true false
```

💡 Pro tip

The "one dot per line" slogan is misleading. The real question is whether the caller must know the internal structure of what it navigates.

© TechTribers. For personal use only. Do not redistribute.

Q438 OP-011 • Advanced

What is "Tell, Don't Ask", and why is an anemic domain model an anti-pattern?

⚡ Quick answer

Tell, Don't Ask: tell an object what to do instead of asking for its data and deciding yourself. An **anemic domain model** is the opposite: classes are bags of getters/setters and all rules sit in service classes, so nothing protects an object's invariants.

With an anemic model the same rule gets duplicated in every service that touches the object, and one forgetful caller breaks the system. A rich model puts the rule next to the data it protects.

Pragmatism: JPA entities, DTOs and records are legitimately data holders. The anti-pattern is when your **core business objects** have no behaviour at all.

Java

```
// Anemic: data holder + rules elsewhere. Any code can put it into an invalid state
class AnemicOrder { String status = "NEW"; }
class OrderService {
    void cancel(AnemicOrder o) { if (o.status.equals("SHIPPED")) throw new
IllegalStateException("shipped"); o.status = "CANCELLED"; }
}
// Rich: the object protects its own invariants
class RichOrder {
    private String status = "NEW";
    void ship() { status = "SHIPPED"; }
    void cancel() { if (status.equals("SHIPPED")) throw new IllegalStateException("already
shipped"); status = "CANCELLED"; }
    String status() { return status; }
}
RichOrder o = new RichOrder();
o.ship();
try { o.cancel(); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }

AnemicOrder a = new AnemicOrder();
a.status = "SHIPPED"; a.status = "CANCELLED";          // nothing stops this bypass
System.out.println(a.status);
```

▶ Output

```
already shipped
CANCELLED
```

⚠️ Interview trap

Getters and setters on every field are not encapsulation, they are public fields with extra steps. Expose operations (`cancel()`, `ship()`), not raw state.

© TechTribers. For personal use only. Do not redistribute.

Q439 OP-012 • Basic

What are DRY, KISS and YAGNI, and when do they conflict?

⚡ Quick answer

DRY (Don't Repeat Yourself): every piece of knowledge has one authoritative place. **KISS** (Keep It Simple): prefer the simplest solution that works. **YAGNI** (You Aren't Gonna Need It): don't build features or abstractions until they are needed. They conflict when removing duplication adds complexity.

Principle	Say it as	Typical violation
DRY	One source of truth	Same validation copied into five controllers
KISS	Simplest thing that works	Factory + strategy + visitor for a two-case `if`
YAGNI	Build it when you need it	A plugin system for a feature with one implementation

Rule of three: tolerate duplication twice, extract on the third occurrence, when you can see what actually varies. Duplication is cheaper than the *wrong* abstraction.

Java

```java
// DRY: the GST rule lives in exactly one place
static double withGst(double price) { return price * 1.18; }
System.out.printf("%.2f %.2f%n", withGst(100), withGst(250));

// WRONG kind of DRY: two unrelated concepts that merely look alike get forced into one abstraction
static double discountOrShipping(double base, boolean isDiscount) { return isDiscount ? base * 0.9 :
base + 40; }
System.out.println(discountOrShipping(100, true) + " " + discountOrShipping(100, false));
```

▶ Output

```
118.00 295.00
90.0 140.0
```

💡 Pro tip

The second example in the code is the failure mode of over-eager DRY: the boolean flag is a smell that two different behaviours were merged.

© TechTribers. For personal use only. Do not redistribute.

🔁 **Q440** OP-013 • Intermediate

How does polymorphism replace if/else and switch chains?

⚡ **Quick answer**

Instead of branching on a type or status code in many places, give each variant its own class that implements a common interface. The `if/else` ladder disappears, and adding a new variant means adding a class rather than editing every conditional. This is the *Replace Conditional with Polymorphism* refactoring.

Use this when the same conditional on the same discriminator is scattered across several methods. For a single, stable two-way branch, a plain `if` is simpler (KISS). With sealed types, a pattern-matching `switch` gives the compiler-checked alternative when you'd rather keep operations in one place.

Java

```
// Before: every new carrier edits this method (and every other method that switches on carrier)
static double costBefore(String carrier, double kg) {
    if (carrier.equals("STANDARD")) return 40 + kg * 10;
    else if (carrier.equals("EXPRESS")) return 90 + kg * 18;
    throw new IllegalArgumentException(carrier);
}
// After: each carrier owns its rule
interface Carrier { double cost(double kg); }
record Standard() implements Carrier { public double cost(double kg) { return 40 + kg * 10; } }
record Express() implements Carrier { public double cost(double kg) { return 90 + kg * 18; } }
record Drone() implements Carrier { public double cost(double kg) { return 150 + kg * 30; } }   // added without editing old code

List<Carrier> carriers = List.of(new Standard(), new Express(), new Drone());
carriers.forEach(c -> System.out.println(c.getClass().getSimpleName() + " " + c.cost(2)));
System.out.println(costBefore("EXPRESS", 2));
```

▶ **Output**

```
Standard 60.0
Express 126.0
Drone 210.0
126.0
```

💡 **Pro tip**

Smell to look for: `switch (type)` or `if (x instanceof ...)` repeated in more than one method. That's the discriminator that wants to become a class hierarchy.

© TechTribers. For personal use only. Do not redistribute.

Q441 OP-014

• Intermediate

What is dependency injection, and how is it different from inversion of control?

⚡ Quick answer

Inversion of Control is the broad principle: your code doesn't control the flow or the object creation, a framework or caller does. **Dependency Injection** is one technique: an object receives its collaborators (usually via the constructor) instead of creating them. **Dependency Inversion** (the D in SOLID) is a design rule: depend on abstractions.

Spring's container is an IoC container: it builds beans and injects them. Injection styles: **constructor** (preferred: immutable, required dependencies, easy tests), **setter** (optional dependencies), **field** (`@Autowired` on a field: convenient but hides dependencies and blocks plain-Java testing).

DI is not a Spring feature. The code above does it with no framework.

Java

```
interface Clock { long now(); }
class SystemClock implements Clock { public long now() { return 1_700_000_000L; } }   // fixed value
so the demo is deterministic
class FixedClock implements Clock {
    private final long t; FixedClock(long t) { this.t = t; }
    public long now() { return t; }
}
class TokenService {
    private final Clock clock;
    TokenService(Clock clock) { this.clock = clock; }                                  // constructor
injection
    boolean expired(long issuedAt, long ttl) { return clock.now() > issuedAt + ttl; }
}
TokenService prod = new TokenService(new SystemClock());
TokenService test = new TokenService(new FixedClock(1_000));                          // test
controls time
System.out.println(prod.expired(1_699_999_000L, 500) + " " + test.expired(900, 500) + " " +
test.expired(900, 50));
```

▶ Output

```
true false true
```

⚠️ Interview trap

Don't mix up the three: IoC (principle) ⊃ DI (technique) and Dependency Inversion (design rule about abstractions). Saying "DI is the same as DIP" is the common slip.

© TechTribers. For personal use only. Do not redistribute.

Section 02

SOLID Principles

Five design principles with a before-and-after example each. All the code runs.

Questions 442–447 · 6 questions



Q442 OP-015

• Intermediate

What are the SOLID principles? Give a one-line explanation of each.

⚡ Quick answer

Single Responsibility: one reason to change. Open/Closed: open for extension, closed for modification. Liskov Substitution: subtypes must be usable wherever the parent is. Interface Segregation: many small interfaces beat one fat one. Dependency Inversion: depend on abstractions, not concretions.

Principle	Question to ask	Smell when violated
Single Responsibility	How many reasons does this class have to change?	God class, huge `Manager` / `Service`
Open/Closed	Can I add a variant without editing existing code?	Growing `if/else` / `switch` on type
Liskov Substitution	Can every subtype replace the parent without surprising callers?	`instanceof` checks, overrides that throw `UnsupportedOperationException`
Interface Segregation	Is every implementer forced to implement methods it doesn't need?	Empty or throwing method implementations
Dependency Inversion	Does high-level policy depend on low-level detail?	`new ConcreteClass()` deep inside business logic

The next five questions take one principle each, with code. They are guidelines for managing change, not laws: applying all five to a 30-line script is over-engineering (KISS and YAGNI still apply).



© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> In an interview, name the **smell** first and then the principle: "This switch grows with every payment type, so I'd apply Open/Closed with a strategy per type." That shows judgement, not memorisation.

1️⃣ **Q443** `OP-016` • Intermediate

Explain the Single Responsibility Principle with an example.

> ⚡ **Quick answer**
> A class should have **one reason to change**, meaning one stakeholder or concern. If tax rules, report formatting and database storage all live in one class, any of those three changes forces edits (and re-testing) of the same file.

"Responsibility" means *a reason to change*, not "one method". A class can have many methods and still be cohesive if they all serve one concern. Splitting too far produces dozens of one-method classes that are hard to navigate.

Java

```java
record Invoice(String id, double amount) { }

// Violates SRP: three reasons to change (tax rules, report format, storage)
class InvoiceGodClass {
    double total(Invoice i) { return i.amount() * 1.18; }
    String print(Invoice i) { return i.id() + ":" + total(i); }
    void save(Invoice i) { /* JDBC code here */ }
}
// Follows SRP: each class has one job
class TaxCalculator { double total(Invoice i) { return i.amount() * 1.18; } }
class InvoicePrinter {
    private final TaxCalculator tax;
    InvoicePrinter(TaxCalculator tax) { this.tax = tax; }
    String print(Invoice i) { return i.id() + ": " + String.format("%.2f", tax.total(i)); }
}
interface InvoiceRepository { void save(Invoice i); }

System.out.println(new InvoicePrinter(new TaxCalculator()).print(new Invoice("INV-1", 1000)));
```

▶ **Output**

```
INV-1: 1180.00
```

> ⚠️ **Interview trap**
> SRP is about stakeholders, not size. A 300-line parser with one concern is fine; a 40-line class mixing validation, pricing and email is not.

© TechTribers. For personal use only. Do not redistribute.

2️⃣ Q444 OP-017

• Intermediate

Explain the Open/Closed Principle with an example.

> ⚡ **Quick answer**
> Software entities should be **open for extension but closed for modification**: you add new behaviour by writing new code (a new class implementing an interface), not by editing tested code. The usual mechanism is polymorphism through an interface or abstract class.

The anti-pattern is `if (type == FESTIVAL) ... else if (type == MEMBER) ...` inside `Checkout`: every new discount edits the class, risking regressions in the old ones. Spring examples: adding a new `HandlerInterceptor`, `Converter` or `@Bean` extends behaviour without changing the framework.

Java

```
interface DiscountRule { double apply(double price); }
class NoDiscount implements DiscountRule { public double apply(double p) { return p; } }
class FestivalDiscount implements DiscountRule { public double apply(double p) { return p * 0.8; } }
class MemberDiscount implements DiscountRule { public double apply(double p) { return p - 50; } }
// NEW: added without editing Checkout

class Checkout {                                        // closed for modification: never changes
when a rule is added
    double total(double price, DiscountRule rule) { return rule.apply(price); }
}
Checkout checkout = new Checkout();
for (DiscountRule r : List.of(new NoDiscount(), new FestivalDiscount(), new MemberDiscount()))
    System.out.println(checkout.total(500, r));
```

▶ Output

```
500.0
400.0
450.0
```

> 💡 **Pro tip**
> OCP is not "never edit a class". Fix bugs freely. It applies to *anticipated axes of change*: apply it after the second or third variant shows you which axis actually varies.

3️⃣ Q445 OP-018

• Advanced

Explain the Liskov Substitution Principle with an example.

> ⚡ **Quick answer**
> Objects of a subtype must be usable anywhere the parent type is expected **without breaking the caller's expectations**. A subclass may not strengthen preconditions, weaken postconditions, or break invariants the parent promised. The classic violation is a `Square` that extends `Rectangle`.

© TechTribers. For personal use only. Do not redistribute.

In math a square *is* a rectangle, but in code the question is about **behaviour**, not taxonomy. A mutable `Rectangle` promises that width and height change independently; `Square` can't keep that promise.

Practical LSP checks: overriding a method to throw `UnsupportedOperationException`, adding `instanceof` checks before using a parent reference, or tightening argument validation in a subclass are all signs of a violation. The JDK itself bends it in places (`Collections.unmodifiableList` throws on `add`), which is why `List` documents optional operations.

Java

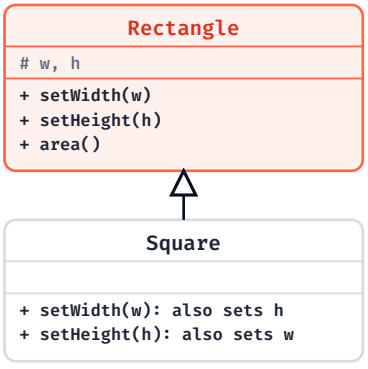
```
class Rectangle {
    protected int w, h;
    Rectangle(int w, int h) { this.w = w; this.h = h; }
    void setWidth(int w) { this.w = w; }
    void setHeight(int h) { this.h = h; }
    int area() { return w * h; }
}
class Square extends Rectangle {
    Square(int s) { super(s, s); }
    @Override void setWidth(int w) { this.w = w; this.h = w; }        // keeps it square, but
changes the contract
    @Override void setHeight(int h) { this.w = h; this.h = h; }
}
static int stretch(Rectangle r) { r.setWidth(5); r.setHeight(4); return r.area(); }   // caller
expects 5 x 4 = 20
System.out.println(stretch(new Rectangle(1, 1)) + " " + stretch(new Square(1)));      // 20 vs 16:
the substitution broke

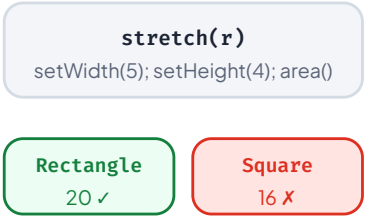
// Fix: stop sharing mutable setters; share a read-only abstraction instead
interface Shape { int area(); }
record Rect(int w, int h) implements Shape { public int area() { return w * h; } }
record Sq(int side) implements Shape { public int area() { return side * side; } }
System.out.println(new Rect(5, 4).area() + " " + new Sq(4).area());
```

▶ Output

```
20 16
20 16
```

Violation: a Square is not a substitutable Rectangle





Fix

```
interface Shape { area() }
record Rect(w, h)
record Sq(side)
```

no shared mutable setters

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> LSP is why `equals`, `compareTo` and `hashCode` contracts matter: a subclass that breaks them breaks every collection that relies on them.

4️⃣ **Q446** `OP-019` • Intermediate

Explain the Interface Segregation Principle with an example.

> ⚡ **Quick answer**
> Clients shouldn't be forced to depend on methods they don't use. Split a fat interface into small, role-focused ones so each implementer only provides what it can actually do. The smell is an implementation with empty or exception-throwing methods.

JDK examples of good segregation: `Readable`, `Closeable`, `AutoCloseable`, `Comparable`, `RandomAccess` are tiny, single-purpose interfaces that classes combine. A counter-example is `java.util.List`: its optional mutators force immutable lists to throw at runtime.

Java 8 default methods ease evolution but don't replace segregation: a default that throws is still a violation in disguise.

Java

```java
// Fat interface (violation): Robot would be forced to implement eat()
interface FatWorker { void work(); void eat(); }

// Segregated roles
interface Workable { void work(); }
interface Eatable { void eat(); }

class Human implements Workable, Eatable {
    public void work() { System.out.println("human works"); }
    public void eat() { System.out.println("human eats"); }
}
class Robot implements Workable {                    // not forced to fake an eat() method
    public void work() { System.out.println("robot works"); }
}
List<Workable> crew = List.of(new Human(), new Robot());
crew.forEach(Workable::work);
```

▶ **Output**

```
human works
robot works
```

> ⚠️ **Interview trap**
> Don't over-split into one-method interfaces for everything. Group methods that always change together and are always used together.

© TechTribers. For personal use only. Do not redistribute.

5️⃣ **Q447** OP-020 • Intermediate

Explain the Dependency Inversion Principle with an example.

> ⚡ **Quick answer**
> High-level policy (business logic) shouldn't depend on low-level details (a specific gateway, database or library). Both should depend on an **abstraction**, and the detail implements it. The *ownership* of the interface sits with the high-level module, so the dependency direction is inverted.

Benefits: swap providers by changing wiring, not business logic; unit-test `CheckoutService` with a lambda or mock; keep vendor SDK types out of the core domain.

DIP is the principle; **dependency injection** is how you usually supply the implementation, and an **IoC container** (Spring) automates the wiring.

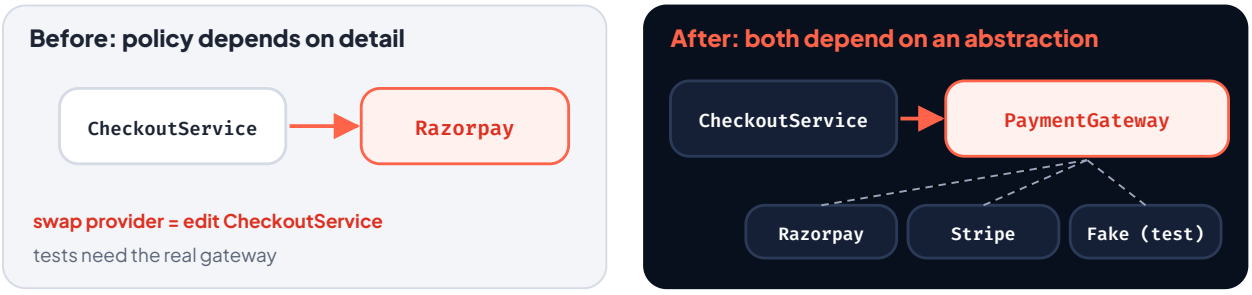
Java

```
interface PaymentGateway { String charge(double amount); }
class Razorpay implements PaymentGateway { public String charge(double a) { return "razorpay:" + a;
} }
class Stripe implements PaymentGateway { public String charge(double a) { return "stripe:" + a; } }

class CheckoutService {                                  // high-level policy knows only the
abstraction
    private final PaymentGateway gateway;
    CheckoutService(PaymentGateway gateway) { this.gateway = gateway; }
    String pay(double amount) { return gateway.charge(amount); }
}
System.out.println(new CheckoutService(new Razorpay()).pay(499));
System.out.println(new CheckoutService(new Stripe()).pay(499));
System.out.println(new CheckoutService(a -> "fake:" + a).pay(1));   // test double, no network
needed
```

▶ **Output**

```
razorpay:499.0
stripe:499.0
fake:1.0
```



© TechTribers. For personal use only. Do not redistribute.

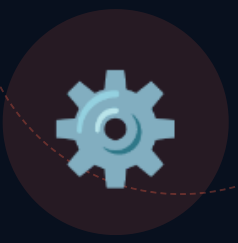
💡 **Pro tip**
Hexagonal / clean architecture is DIP at system scale: the domain defines ports (interfaces), and adapters (web, DB, messaging) implement them.

Section 03

Inheritance & Polymorphism: How It Really Works

Constructors, initialisation order, binding, overloading, casting and the JVM's dispatch rules, including the output-prediction questions interviewers love.

Questions 448–461 · 14 questions



🌳 Q448 OP-021 • Basic

What types of inheritance does Java support?

⚡ **Quick answer**
Classes support **single**, **multilevel** and **hierarchical** inheritance (each class extends exactly one parent). **Multiple inheritance of type** is achieved with interfaces; a class may implement many. Multiple inheritance of *state* through classes is not allowed, which avoids the diamond problem.

Why no multiple class inheritance? If `C extends A, B` and both define `void f()` or a field `x`, which one does `C` inherit? Interfaces avoid the ambiguity because they have no instance state, and when two defaults clash the compiler forces the class to override and choose explicitly (`Swimmer.super.move()`).

Java

```
class Animal { String eat() { return "eats"; } }
class Mammal extends Animal { String feed() { return "milk"; } }    // single:       Mammal -> Animal
class Dog extends Mammal { }                                        // multilevel:   Dog -> Mammal -> Animal
class Cat extends Mammal { }                                        // hierarchical: Dog and Cat share Mammal

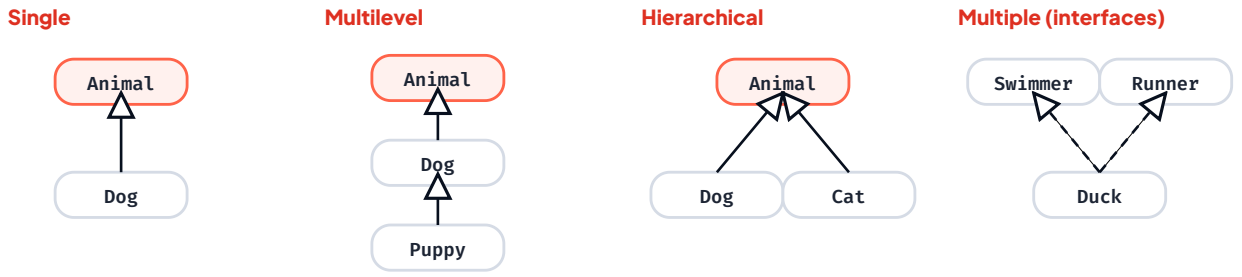
interface Swimmer { default String move() { return "swims"; } }
interface Runner { default String move() { return "runs"; } }
class Duck extends Animal implements Swimmer, Runner {              // multiple (via interfaces)
    public String move() { return Swimmer.super.move() + " and " + Runner.super.move(); }   // conflict must be resolved
}
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(new Dog().eat() + " " + new Cat().feed() + " | " + new Duck().eat() + ", " + new Duck().move());
```

▶ Output

```
eats milk | eats, swims and runs
```



Java: classes extend ONE class; interfaces give multiple inheritance of type (and default behaviour).

⚠️ Interview trap

"Hybrid inheritance" is not a separate Java feature; it's just a mix of the above using interfaces.

Q449 OP-022 • Basic

Are constructors inherited? Can they be overridden?

⚡ Quick answer

No to both. Constructors are **not inherited** and can't be overridden, because they aren't members and their name is the class name. A subclass must define its own constructor and call a parent constructor with `super(...)`, explicitly or implicitly.

Rules worth remembering: a constructor can't be `static`, `final`, `abstract` or `synchronized`; every constructor starts with `this(...)` or `super(...)` (an implicit `super()` is inserted otherwise); and if the parent has no no-arg constructor, every subclass constructor must call one explicitly.

Java

```
class Base {
    Base(int x) { System.out.println("Base(" + x + ")"); }
}
class Derived extends Base {
    // Derived(int x) is NOT inherited from Base. With no constructor at all this class would not compile,
    // because the implicit super() finds no Base() to call.
    Derived() { super(7); System.out.println("Derived()"); }
}
new Derived();
// new Derived(5);   // compile error: constructor Derived in class Derived cannot be applied to given types
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
Base(7)
Derived()
```

⚠️ **Interview trap**

Constructors can be *overloaded* (same name, different parameters) but never overridden. Mixing the two terms up is a frequent interview slip.

Q450 OP-023 • Intermediate

In what order are static blocks, instance blocks and constructors executed in an inheritance chain?

⚡ **Quick answer**

First the **static** initialisers of the parent and then the child (once per class, when it is first used). Then, for each object created: the parent's instance initialisers and constructor body, then the child's. Parents always initialise before children.

Field initialisers and instance blocks run in **source order** within a class, just before the constructor body (right after the `super(...)` call). Static state is initialised once when the class is initialised, which happens on first `new`, static method call or non-constant static field access.

Java

```
class Parent {
    static { System.out.println("1 Parent static"); }
    { System.out.println("3 Parent instance block"); }
    Parent() { System.out.println("4 Parent constructor"); }
}
class Child extends Parent {
    static { System.out.println("2 Child static"); }
    { System.out.println("5 Child instance block"); }
    Child() { super(); System.out.println("6 Child constructor"); }
}
new Child();
System.out.println("--- second object");
new Child();
```

▶ Output

```
1 Parent static
2 Child static
3 Parent instance block
4 Parent constructor
5 Child instance block
6 Child constructor
--- second object
3 Parent instance block
4 Parent constructor
```


© TechTribers. For personal use only. Do not redistribute.

```
5 Child instance block
6 Child constructor
```

1. **Parent static blocks** — once, when Parent is first used
2. **Child static blocks** — once, when Child is first used
3. **Parent instance blocks + field initialisers** — every new Child()
4. **Parent constructor body**
5. **Child instance blocks + field initialisers**
6. **Child constructor body**

Static steps (1–2): once per class
Instance steps (3–6): once per object, parent always before child

💡 **Pro tip**
Whenever you see "predict the output" with `static`, `{}` and constructors, write the numbered order for the first object, then note that static steps don't repeat for the second.

⚠️ **Q451** OP-024 • Advanced

What happens if a constructor calls an overridable method?

⚡ **Quick answer**
The overridden version in the **subclass** runs *before* the subclass's own fields and constructor have been initialised, so it sees default values (`null` , `0` , `false`). This is a classic source of `NullPointerException` and is why you shouldn't call overridable methods from constructors.

Sequence: `new SalesReport()` → `Report()` runs first → it calls `title()` → dynamic dispatch picks `SalesReport.title()` → the `name` and `quarter` fields of `SalesReport` haven't been assigned yet → `null null`.

How to avoid it: call only `private` , `static` or `final` methods from constructors, do the work in a separate `init()` called after construction, use a factory method, or (Java 25) assign subclass fields before `super(...)` with flexible constructor bodies.

Java

```
class Report {
    Report() { System.out.println("header: " + title()); }        // calls an overridable method
    String title() { return "generic"; }
}
class SalesReport extends Report {
    private String name = "Sales";                                 // assigned AFTER Report()
finishes
    private String quarter = "Q3";
    @Override String title() { return name + " " + quarter; }
```


© TechTribers. For personal use only. Do not redistribute.

```
}
new SalesReport();
```

▶ Output

```
header: null null
```

⚠️ **Interview trap**

Same hazard with instance initialisation of fields: even `final` fields can be observed in their default state if the parent's constructor calls into the child.

Q452 OP-025 • Intermediate

What is the difference between static (compile-time) and dynamic (runtime) binding?

⚡ **Quick answer**

Static binding is resolved by the compiler from *declared* types: overloaded method signatures, `static`, `private` and `final` methods, and fields. **Dynamic binding** is resolved at runtime from the object's *actual* class: overridden instance methods. Dynamic binding is what makes polymorphism work.

Two-step rule for method calls: 1. **Compile time**: use the *declared* type of the reference and the argument types to choose the method *signature*. 2. **Runtime**: if that method is an overridable instance method, call the version in the object's *actual* class.

That is why `a.describe(new Dog())` prints `animal-arg`: step 1 only sees `Animal.describe(Animal)`, and `Dog` never overrode it.

Java

```
class Animal {
    String sound() { return "..."; }
    static String kind() { return "animal"; }
    String describe(Animal a) { return "animal-arg"; }
}
class Dog extends Animal {
    @Override String sound() { return "woof"; }
    static String kind() { return "dog"; }                  // hides, does not override
    String describe(Dog d) { return "dog-arg"; }            // OVERLOADS describe(Animal), does not
override it
}
Animal a = new Dog();
System.out.println(a.sound());                              // dynamic: actual type Dog
System.out.println(a.kind());                               // static: declared type Animal
System.out.println(a.describe(new Dog()));                  // overload chosen at compile time from
Animal's methods
System.out.println(((Dog) a).describe(new Dog()));          // with a Dog reference the more specific
overload is visible
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
woof
animal
animal-arg
dog-arg
```

Compile time (static / early binding)

- overloaded method SIGNATURE chosen
- from the declared (reference) types
- static, private and final methods
- field access (fields are not polymorphic)
- constructors

Run time (dynamic / late binding)

- overridden INSTANCE method chosen
- from the actual object's class
- via the vtable (invokevirtual)
- or the itable (invokeinterface)
- this is what makes polymorphism work

`Animal a = new Dog(); a.sound() → runtime type Dog · a.kind() (static) → declared type Animal`

⚠️ **Interview trap**

Overloading is NOT polymorphism in the runtime sense: the overload is fixed at compile time. Only overriding is dispatched dynamically.

Q453 OP-026 • Advanced

How does the compiler choose between overloaded methods?

⚡ **Quick answer**

Three phases, stopping at the first that finds a match: (1) **widening** primitive conversion or subtyping without boxing, (2) **boxing/unboxing**, (3) **varargs**. Within a phase, the **most specific** applicable method wins; if none is most specific the call is ambiguous.

Notes on the results:

- `byte`, `short` and `char` widen to `int`, so `f(int)` wins (phase 1) even though `Integer` exists.
- `float` can't widen to `int` or `long`, so phase 1 fails; phase 2 boxes it to `Float`, which is an `Object`.
- `g(null)` is legal because `String` is more specific than `Object`. With `g(Object)` and `g(Integer)` and `g(String)` together, `g(null)` would be **ambiguous**.

Java

```
static String f(int x) { return "int"; }
static String f(long x) { return "long"; }
static String f(Integer x) { return "Integer"; }
static String f(Object x) { return "Object"; }
static String f(int... x) { return "varargs"; }
```


© TechTribers. For personal use only. Do not redistribute.

```
static String g(Object o) { return "Object"; }
static String g(String s) { return "String"; }

byte b = 1; short s = 2; char c = 'c'; long l = 3; float fl = 1f;
System.out.println(f(b) + " " + f(s) + " " + f(c) + " " + f(5) + " " + f(l));    // widening to int/long beats boxing
System.out.println(f(Integer.valueOf(7)) + " " + f("text") + " " + f() + " " + f(fl));
System.out.println(g(null) + " " + g((Object) "x"));                              // most specific wins, the cast picks Object
```

▶ Output

```
int int int int long
Integer Object varargs Object
String Object
```

⚠️ Interview trap

Varargs is always the last resort, so adding `f(int...)` never changes calls that already match something else. But overload sets that mix boxing and varargs (`f(Object...)` vs `f(Integer...)`) produce surprising ambiguity errors.

Q454 OP-027 • Intermediate

Are fields polymorphic? What is field hiding?

⚡ Quick answer

No. Only instance **methods** are dispatched dynamically. Field access is resolved at compile time from the declared type, so a subclass field with the same name merely **hides** the parent's field; both exist in the object.

One `Child` object contains **two** separate `name` fields. `p.name` uses the declared type `Parent`, so it reads `Parent.name`; `p.name()` is virtual and runs `Child.name()`, which reads `Child.name`.

Java

```
class Parent {
    String name = "parent";
    String name() { return name; }
}
class Child extends Parent {
    String name = "child";                    // hides Parent.name, it does not override it
    @Override String name() { return name; }  // methods ARE polymorphic
}
Parent p = new Child();
System.out.println(p.name + " " + p.name() + " " + ((Child) p).name + " " + ((Parent) p).name);
```

▶ Output

```
parent child child parent
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Avoid field hiding entirely: use different names or, better, keep fields private and expose behaviour through methods, which dispatch correctly.

📋 Q455 OP-028 • Intermediate

What are the rules for overriding a method (access, return type, exceptions)?

> ⚡ **Quick answer**
> The overriding method must have the **same name and parameter types**; its access can only be **equal or wider**; its return type must be the **same or a subtype** (covariant return); and it may throw **no new or broader checked exceptions**. Static, private and final methods can't be overridden; `@Override` makes the compiler check it.

Aspect	Allowed in override	Compile error example
Access	Same or wider (`protected` → `public`)	`public` → `protected`
Return type	Same or subtype (`Number` → `Integer`)	`Integer` → `Number` ; `int` → `long`
Checked exceptions	Same, narrower, or none	Adding `throws Exception`
Unchecked exceptions	Any	(always allowed)
`static` / `private` / `final`	Not overridable (hiding or error)	Overriding a `final` method

The reasoning is **substitutability** (LSP): code written against the parent must still compile and work with the child, so a child can't demand more or promise less.

Java

```
class Source {
    protected Number read() throws java.io.IOException { return 1; }
}
class FastSource extends Source {
    @Override public Integer read() { return 42; }        // access widened, covariant return, checked exception dropped
}
Source r = new FastSource();
try { System.out.println(r.read()); } catch (java.io.IOException e) { }
```

▶ Output

```
42
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Always write `@Override`. A typo in the method name or parameter type then fails to compile instead of silently creating a new overload.

⬆️ **Q456** `OP-029` • Basic

What are upcasting and downcasting, and when does ClassCastException occur?

> ⚡ **Quick answer**
> **Upcasting** (child → parent) is implicit and always safe. **Downcasting** (parent → child) needs an explicit cast and is checked at runtime: if the object's actual class isn't the target (or a subtype), the JVM throws `ClassCastException`. Guard it with `instanceof`.

The compiler only rejects casts between **unrelated** types (`(String) new Dog()` won't compile). Between related types it trusts you and defers the check to runtime. Frequent downcasting is a design smell: it usually means the parent interface is missing an operation.

Java

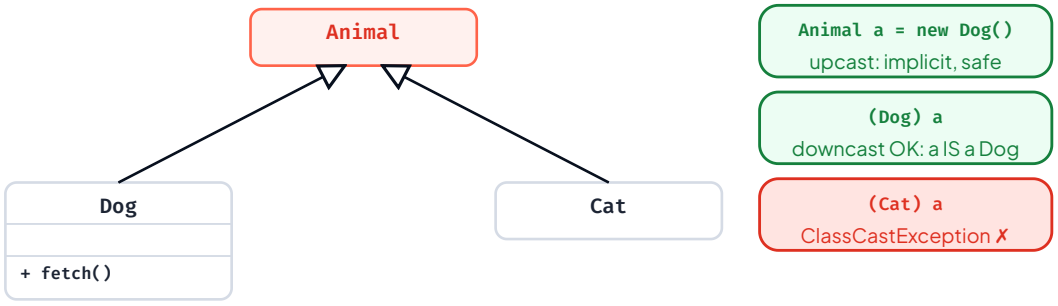
```
class Animal { }
class Dog extends Animal { void fetch() { System.out.println("fetch"); } }
class Cat extends Animal { }

Animal a = new Dog();                    // upcast: implicit
if (a instanceof Dog d) d.fetch();       // safe downcast with pattern matching (Java 16+)
try {
    Cat c = (Cat) a;                     // compiles (Cat is an Animal) but a is really a Dog
} catch (ClassCastException e) {
    System.out.println(e.getClass().getSimpleName());
}
Object o = "text";
System.out.println(o instanceof String s && s.length() == 4);
```

▶ **Output**

```
fetch
ClassCastException
true
```


© TechTribers. For personal use only. Do not redistribute.



Guard downcasts with instanceof (pattern matching): if (a instanceof Dog d) d.fetch();

⚠️ **Interview trap**

`instanceof` is `false` for `null`, so `null instanceof Dog` never throws and `(Dog) null` also succeeds, giving `null`.

Q457 OP-030 • Advanced

Can a subclass in another package access a protected member through a parent reference?

⚡ **Quick answer**

No. A subclass in a different package may use a `protected` member only through a reference of **its own type (or a subtype)**, including `this`. Calling it on a *parent-typed* reference is a compile error, even though the subclass inherits the member.

Why the rule exists: otherwise any subclass could reach into *other* subclasses' protected state through a common parent reference, defeating the idea that `protected` is for implementing inheritance. (Within the **same package**, `protected` also behaves like package-private, so this restriction only appears across packages.)

Access levels summary for reference: `public` > `protected` (subclasses + package) > package-private > `private`.

Java

```java
// package a;
public class Base {
    protected String hello() { return "hello from Base"; }
}

// package b;
public class Sub extends Base {
    String test(Sub me, Base other) {
        String x = hello();          // OK: inherited member on this
        String y = me.hello();       // OK: reference of the subclass type
        String z = other.hello();    // ERROR: reference of the PARENT type
        return x + y + z;
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
}

// javac:
// b/Sub.java:7: error: hello() has protected access in Base
//         String z = other.hello();    // reference of the PARENT type
```

> 💡 **Pro tip**
> The error above was produced by compiling the two files with JDK 25. If an interviewer asks for a "protected tricky question", this is almost always it.

Q458 OP-031 • Advanced

How does the JVM implement method calls and dynamic dispatch?

> ⚡ **Quick answer**
> Bytecode has five call instructions: `invokevirtual` (instance method via the object's class, using a **vtable**), `invokeinterface` (via the interface method table), `invokespecial` (constructors, `super.` calls), `invokestatic`, and `invokedynamic` (lambdas, string concatenation). Overriding works because each class's vtable slot points to its own implementation.

How `a.sound()` works for `Animal a = new Dog()`: `invokevirtual` loads the receiver's class (`Dog`), looks at its vtable slot for `sound()`, finds `Dog.sound`, and jumps there. The slot number is fixed at link time, so it's a cheap indexed lookup, and the JIT can often inline it entirely when only one implementation is loaded.

`invokeinterface` needs a slower lookup because a class can implement many interfaces, but HotSpot caches it with inline caches.

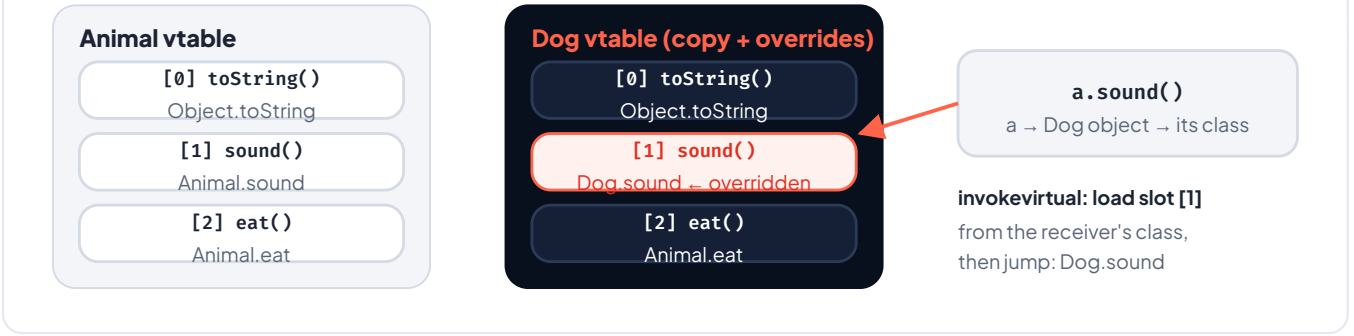
Java

```java
interface Greeter { String hi(); }
class Dispatch {
    String name() { return "n"; }
    private String secret() { return "s"; }
    static String util() { return "u"; }
    String run(Greeter g) {
        name();                 // virtual call
        secret();               // private
        util();                 // static
        g.hi();                 // interface call
        return super.toString();   // super call
    }
}

// javap -c -p Dispatch  (JDK 25) -- the call instructions in run():
//    1: invokevirtual   Dispatch.name:()Ljava/lang/String;
//    6: invokevirtual   Dispatch.secret:()Ljava/lang/String;      <- private methods use
invokevirtual since Java 11 (nestmates)
//   10: invokestatic    Dispatch.util:()Ljava/lang/String;
//   15: invokeinterface Greeter.hi:()Ljava/lang/String;
```


© TechTribers. For personal use only. Do not redistribute.

```
//   22: invokespecial   java/lang/Object.toString:()Ljava/lang/String;      <- super call is non-virtual
```



💡 **Pro tip**

The output above is real `javap` output, not typed from memory. Run `javap -c -p YourClass` to see which instruction your own calls compile to.

Q459 OP-032 • Advanced

How do equals() and hashCode() behave in an inheritance hierarchy?

⚡ **Quick answer**

The `equals` contract needs reflexivity, **symmetry**, transitivity and consistency. With inheritance it is hard to satisfy all of them: using `instanceof` lets a subclass break symmetry; using `getClass()` keeps symmetry but violates Liskov substitution. There is no perfect solution when a subclass adds state that participates in equality.

The robust answers (Effective Java, Item 10): **favour composition** (a `ColorPoint` *has a* `Point` and exposes `asPoint()`), make the class `final`, **or use a** record**. Also remember: whenever you override `equals`, override `hashCode` consistently, or hash-based collections break.

Java

```
class Point {
    final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    @Override public boolean equals(Object o) { return o instanceof Point p && p.x == x && p.y == y;
}
    @Override public int hashCode() { return Objects.hash(x, y); }
}
class ColorPoint extends Point {
    final String color;
    ColorPoint(int x, int y, String c) { super(x, y); color = c; }
    @Override public boolean equals(Object o) { return o instanceof ColorPoint cp && super.equals(o)
&& cp.color.equals(color); }
}
Point p = new Point(1, 2);
ColorPoint cp = new ColorPoint(1, 2, "red");
```

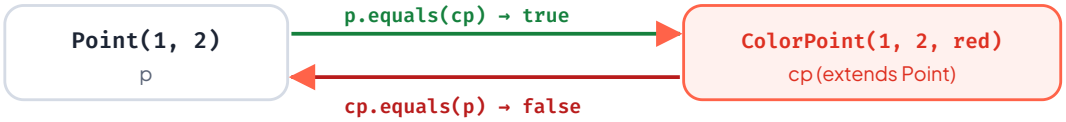

© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(p.equals(cp) + " " + cp.equals(p));          // true false: symmetry is broken

// getClass() version: symmetric, but a harmless subclass is no longer equal to its parent (violates LSP)
class Point2 {
    final int x, y;
    Point2(int x, int y) { this.x = x; this.y = y; }
    @Override public boolean equals(Object o) { return o != null && getClass() == o.getClass() && ((Point2) o).x == x && ((Point2) o).y == y; }
    @Override public int hashCode() { return Objects.hash(x, y); }
}
Point2 anonymousSubclass = new Point2(1, 2) { };
System.out.println(new Point2(1, 2).equals(anonymousSubclass));
```

▶ Output

```
true false
false
```



Symmetry is broken: x.equals(y) must equal y.equals(x). Prefer composition, final classes or records.

⚠️ **Interview trap**

Generated `equals` from an IDE or Lombok's `@EqualsAndHashCode` on a class hierarchy needs `callSuper` and `canEqual` considerations. Blindly generating it in a subclass hits the same symmetry problem.

Q460 OP-033 • Basic

What methods does the java.lang.Object class provide?

⚡ **Quick answer**

Every class inherits eleven methods from `Object`: `equals`, `hashCode`, `toString`, `getClass`, `clone`, `finalize` (deprecated), and the monitor methods `wait` (three overloads), `notify`, `notifyAll`. Only `equals`, `hashCode`, `toString` and (rarely) `clone` are normally overridden.

Method	Purpose	Override?
`equals(Object)`	Logical equality (default: identity)	Yes, with `hashCode`
`hashCode()`	Hash for `HashMap` / `HashSet`	Yes, with `equals`
`toString()`	Readable representation	Yes (logging, debugging)

© TechTribers. For personal use only. Do not redistribute.

`getClass()`	Runtime class (`final`)	No
`clone()` **(protected)**	Shallow copy; needs `Cloneable`	Rarely; prefer copy constructors
`wait` / `notify` / `notifyAll`	Monitor-based thread coordination (`final`)	No
`finalize()` **(protected)**	Deprecated for removal: use `Cleaner` /try-with-resources	Don't

The listing above omits `clone` and `finalize` because they are `protected`, so `getMethods()` (public only) doesn't return them.

Java

```
Arrays.stream(Object.class.getMethods())              // public methods only
        .map(Method::getName).distinct().sorted()
        .forEach(m -> System.out.print(m + " "));
System.out.println();
System.out.println(Modifier.isFinal(Object.class.getMethod("getClass").getModifiers()) + " "
        + Modifier.isFinal(Object.class.getMethod("notify").getModifiers()));
```

▶ **Output**

```
equals getClass hashCode notify notifyAll toString wait
true true
```

⚠️ **Interview trap**

`getClass()` and the `wait` / `notify` family are `final`: they can't be overridden, which is why monitor semantics are guaranteed for every object.

🏷️ **Q461** `OP-034` • Intermediate

What is a marker interface, and how does it compare with an annotation?

⚡ **Quick answer**

A **marker interface** declares no methods and just tags a class (`Serializable`, `Cloneable`, `RandomAccess`). It creates a *type*, so it can be used in method signatures and checked with `instanceof`. A **marker annotation** (`@FunctionalInterface`, `@Deprecated`) carries metadata read by tools or reflection. Prefer annotations for metadata, marker interfaces when you need a type.

	Marker interface	Marker annotation
Defines a type	Yes: usable as a parameter type	No
Compile-time check	Yes (type system)	Only with annotation processors
Read at runtime	`instanceof`	Reflection (needs `RUNTIME` retention)

© TechTribers. For personal use only. Do not redistribute.

Can apply to methods/fields	No (classes only)	Yes
Examples	`Serializable`, `RandomAccess`	`@Override`, `@FunctionalInterface`

`Serializable` is the textbook marker: `ObjectOutputStream.writeObject` checks `instanceof Serializable` at runtime. `RandomAccess` lets algorithms such as `Collections.binarySearch` pick an index-based or iterator-based strategy.

Java

```
System.out.println((new ArrayList<>() instanceof RandomAccess) + " " + (new LinkedList<>()
instanceof RandomAccess));
System.out.println(Runnable.class.isAnnotationPresent(FunctionalInterface.class));     // marker
annotation, read via reflection

class NoClone { Object copy() throws CloneNotSupportedException { return super.clone(); } }   //
does NOT implement Cloneable
try { new NoClone().copy(); } catch (CloneNotSupportedException e) {
System.out.println("CloneNotSupportedException"); }
```

▶ **Output**

```
true false
true
CloneNotSupportedException
```

💡 **Pro tip**

A marker interface also lets the compiler enforce that only tagged objects are accepted: `void archive(Auditable a)` is checked at compile time, while an annotation would only be checked by a tool or at runtime.

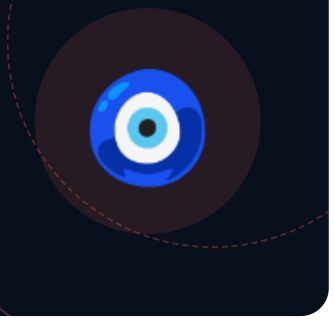
© TechTribers. For personal use only. Do not redistribute.

Section 04

Enums as Objects

Enums are full classes: constants are singleton objects with fields, constructors and behaviour.

Questions 462–464 · 3 questions



Q462 OP-035

• Intermediate

Are enums just constants? What can an enum contain in Java?

⚡ Quick answer

No. An enum is a special **class** whose instances are the named constants. It can have fields, constructors (always private), methods, static members and can implement interfaces. The compiler adds `values()`, `valueOf(String)`, `ordinal()` and `name()`, and each constant exists exactly once.

Properties worth knowing:

- Enums implicitly extend `java.lang.Enum`, so they **can't extend another class**, but they can implement interfaces.
- They are `final` (unless a constant has a body), serialisation-safe and reflection-safe: `new` and cloning are blocked.
- They work in `switch`, in `EnumMap` / `EnumSet`, and as thread-safe singletons.

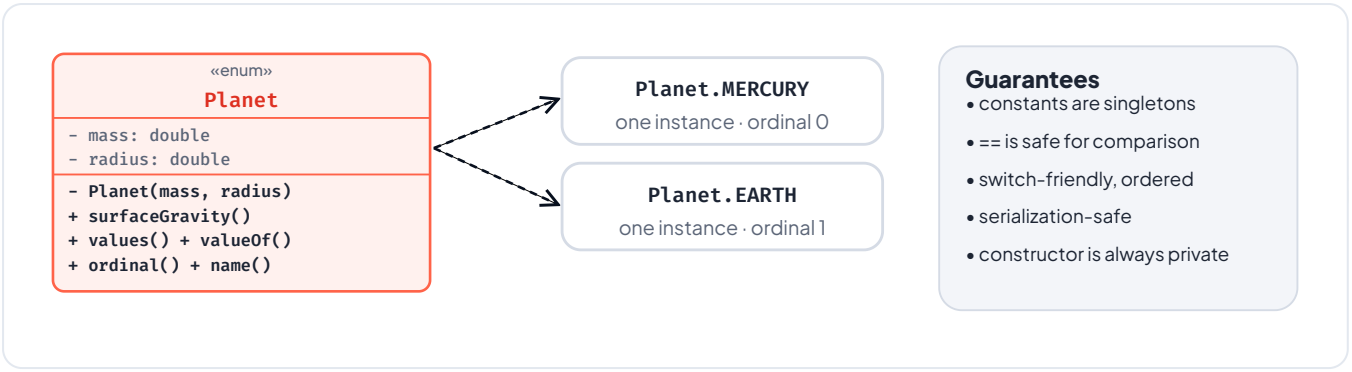
Java

```
enum Planet {
    MERCURY(3.303e+23, 2.4397e6), EARTH(5.976e+24, 6.37814e6);
    private final double mass, radius;                        // enums are classes: fields...
    Planet(double mass, double radius) { this.mass = mass; this.radius = radius; }   // ...and a
private constructor
    double surfaceGravity() { return 6.67300E-11 * mass / (radius * radius); }
}
for (Planet p : Planet.values()) System.out.printf("%s #%d %.2f%n", p, p.ordinal(),
p.surfaceGravity());
System.out.println(Planet.valueOf("EARTH") == Planet.EARTH);   // == is safe: each constant is a
singleton
try { Planet.valueOf("PLUTO"); } catch (IllegalArgumentException e) {
System.out.println("IllegalArgumentException"); }
```

▶ Output

```
MERCURY #0 3.70
EARTH #1 9.80
true
IllegalArgumentException
```


© TechTribers. For personal use only. Do not redistribute.



⚠️ **Interview trap**

`ordinal()` is the position in the declaration. Never persist it (database column, JSON) because reordering constants changes the meaning. Store `name()` or an explicit code field instead.

🎭 **Q463** OP-036 • Intermediate

How do you give each enum constant its own behaviour?

⚡ **Quick answer**

Declare an abstract method (or interface method) in the enum and implement it in a **constant-specific class body** for each constant. This is the *Strategy pattern* built into the language, and it replaces `switch` statements on the enum.

Each constant with a body is an anonymous subclass of the enum. Adding a new plan means adding a constant with its behaviour, and the compiler (via the abstract method) forces you to implement it, which is Open/Closed with compile-time safety. For pure data (name, code, price) plain constructor fields are simpler than bodies.

Java

```
interface Priced { double price(double base); }
enum Plan implements Priced {
    FREE       { public double price(double base) { return 0; } },
    PRO        { public double price(double base) { return base * 0.9; } },
    ENTERPRISE { public double price(double base) { return base * 0.75; }
                 @Override String support() { return "24x7"; } };         // a constant can override
other methods too
    String support() { return "email"; }
}
for (Plan p : Plan.values()) System.out.println(p + " " + p.price(1000) + " " + p.support());
```

▶ **Output**

```
FREE 0.0 email
PRO 900.0 email
ENTERPRISE 750.0 24x7
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

Interview example: operators in a calculator (`PLUS("+") { apply(a, b) }`), HTTP status groups, or state machines where each state constant defines `next()` .

Q464 OP-037 • Intermediate

Why use EnumMap and EnumSet instead of HashMap and HashSet?

⚡ **Quick answer**

`EnumMap` and `EnumSet` are specialised collections for enum keys. They are backed by an **array / bit vector** indexed by ordinal, so they are faster and more memory-efficient than hashing, iterate in the enum's declaration order, and never need `hashCode` / `equals` .

An `EnumSet` of up to 64 constants is a single `long` bit mask, so `contains` , `add` and set operations like `complementOf` are essentially bit operations. Use them for flags, permissions (`EnumSet<Permission>`) and lookups keyed by a status or type.

Java

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

EnumMap<Day, Integer> hours = new EnumMap<>(Day.class);
hours.put(Day.WED, 6);
hours.put(Day.MON, 8);
System.out.println(hours);                                  // ordered by ordinal, not
insertion order

EnumSet<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
System.out.println(weekend + " " + EnumSet.complementOf(weekend) + " " + EnumSet.range(Day.TUE,
Day.THU));
System.out.println(EnumSet.allOf(Day.class).size() + " " + EnumSet.noneOf(Day.class).isEmpty());
```

▶ **Output**

```
{MON=8, WED=6}
[SAT, SUN] [MON, TUE, WED, THU, FRI] [TUE, WED, THU]
7 true
```

⚠️ **Interview trap**

`EnumMap` is not thread-safe and doesn't allow `null` keys (null values are fine). Wrap with `Collections.synchronizedMap` if you share it.

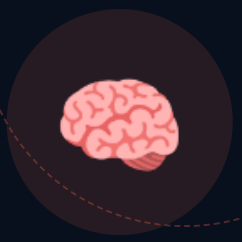
© TechTribers. For personal use only. Do not redistribute.

Section 05

Object Modelling Techniques

Design techniques that senior interviews probe: coding to interfaces, value objects vs entities, factories, type-system variance, double dispatch, null objects and designing for inheritance.

Questions 465–471 · 7 questions



Q465 OP-038

• Intermediate

What does "program to an interface, not an implementation" mean?

⚡ Quick answer

Declare variables, parameters and return types using the most general **abstraction** that meets your needs (`List` , `Map` , `Collection` , your own interface) and keep concrete class names confined to the place where objects are created. Then implementations can be swapped without changing the code that uses them.

Rules of thumb: *accept* the most general type (`Collection<? extends Number>`), *return* a type that's useful but not over-specific (`List<T>` , not `ArrayList<T>`), and never expose implementation types in public APIs. The exception is when you need an implementation-specific contract (`LinkedHashMap` ordering, `TreeMap` navigation).

Java

```
List<String> names = new ArrayList<>();                 // declared type is the interface
names.add("b"); names.add("a");
names = new LinkedList<>(names);                        // swapping the implementation is a one-line change
Collections.sort(names);
System.out.println(names + " " + names.getClass().getSimpleName());

static double total(Collection<? extends Number> nums) { // accept the most general type that works
    double t = 0;
    for (Number n : nums) t += n.doubleValue();
    return t;
}
System.out.println(total(List.of(1, 2.5)) + " " + total(Set.of(3)));
```

▶ Output

```
[a, b] LinkedList
3.5 3.0
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`ArrayList<String> list = new ArrayList<>()` as a *field or parameter type* locks every caller to `ArrayList`. Locally inside one method it's harmless.

Q466 OP-039 • Intermediate

What is the difference between a value object and an entity?

⚡ **Quick answer**

An **entity** has an identity that persists over time (`Customer` C1 is the same customer even if the email changes), so equality is by **id**. A **value object** has no identity and is defined entirely by its attributes (`Money(500, INR)`), so equality is by **value** and it is usually immutable.

This is Domain-Driven Design vocabulary. Practical consequences: make value objects immutable (records), put business rules in them (`Money.plus` checks currencies), and implement `equals` / `hashCode` for entities on the id only, never on mutable fields (which would break `HashSet` membership when state changes).

Java

```
record Money(long paise, String currency) { }                // value object: equal when all
components are equal

class Customer {                                               // entity: identity by id, state
may change
    private final String id;
    private String email;
    Customer(String id, String email) { this.id = id; this.email = email; }
    void changeEmail(String e) { email = e; }
    @Override public boolean equals(Object o) { return o instanceof Customer c && c.id.equals(id); }
    @Override public int hashCode() { return id.hashCode(); }
}
System.out.println(new Money(500, "INR").equals(new Money(500, "INR")));
Customer a = new Customer("C1", "a@x.com"), b = new Customer("C1", "other@x.com");
System.out.println(a.equals(b));                                // same identity even though state
differs
```

▶ **Output**

```
true
true
```

💡 **Pro tip**

Java records are a natural fit for value objects. JPA entities are the natural fit for entities, with `equals` / `hashCode` based on the business key or id.

© TechTribers. For personal use only. Do not redistribute.

Q467 OP-040 • Intermediate

What are static factory methods, and why prefer them to constructors?

⚡ Quick answer

A **static factory method** is a named static method that returns an instance (`List.of` , `Integer.valueOf` , `LocalDate.of`). Unlike constructors it has a **name**, can **return a cached or subtype instance**, and need not create a new object each call. Make the constructor private to force callers through it.

Advantages (Effective Java, Item 1): descriptive names, instance control (caching, singletons, flyweights), can return any subtype (`List.of` returns a private immutable implementation), and can return `Optional` / `null` -free results.

Disadvantages: classes with only private constructors can't be subclassed, and static factories are less discoverable than constructors, so follow naming conventions: `of` , `from` , `valueOf` , `create` , `newInstance` , `getInstance` .

Java

```
final class Temperature {
    private final double celsius;
    private Temperature(double c) { celsius = c; }
    static Temperature ofCelsius(double c) { return new Temperature(c); }                    //
the name says what the number means
    static Temperature ofFahrenheit(double f) { return new Temperature((f - 32) * 5 / 9); }     //
same parameter type, different meaning
    double celsius() { return celsius; }
}
System.out.println(Temperature.ofFahrenheit(212).celsius() + " " +
Temperature.ofCelsius(37).celsius());
System.out.println(Integer.valueOf(100) == Integer.valueOf(100));       // cached instances:
impossible with a constructor
```

▶ Output

```
100.0 37.0
true
```

💡 Pro tip

Two constructors with the same signature can't coexist, but two static factories with different names can. That's often the cleanest fix for `Temperature(double)` ambiguity.

© TechTribers. For personal use only. Do not redistribute.

Q468 OP-041 • Advanced

Why are Java arrays covariant but generics invariant?

⚡ Quick answer

Arrays were covariant from Java 1.0 (`String[]` is an `Object[]`) before generics existed, so the type check had to happen at **runtime** (`ArrayStoreException`). Generics are **invariant** (`List<String>` is not a `List<Object>`), so the compiler catches the mistake at compile time. Wildcards (`? extends`) restore safe flexibility.

Because arrays carry their element type at runtime (reified) and generics erase theirs, `new T[10]` and `new List<String>[10]` are illegal. If `List<String>` were a `List<Object>`, you could add an `Integer` through the `Object` view and corrupt the list, which is exactly the hole covariant arrays leave open.

Java

```
Object[] objs = new String[1];                    // allowed: arrays are covariant
try { objs[0] = 42; }                             // compiles, but the real array only holds String
catch (ArrayStoreException e) { System.out.println("ArrayStoreException at runtime"); }

List<String> strings = new ArrayList<>();
// List<Object> bad = strings;                     // compile error: generics are invariant
List<? extends Object> ok = strings;               // read-only view through a wildcard
System.out.println(ok.size());
```

▶ Output

```
ArrayStoreException at runtime
0
```

⚠️ Interview trap

Covariant arrays are a historical wart. In new code prefer `List<T>`; reserve arrays for performance-critical primitives.

Q469 OP-042 • Advanced

What is the Visitor pattern, and what is double dispatch?

⚡ Quick answer

Java picks the method to call using the **runtime type of one object only** (single dispatch). The Visitor pattern fakes **double dispatch**: `node.accept(visitor)` dispatches on the node's type, and inside, `visitor.visitX(this)` dispatches on the visitor's type. It lets you add operations to a class hierarchy without editing the classes.

Visitor trades one extensibility for another: adding a new **operation** (a new visitor) is easy, but adding a new **node type** forces every visitor to change. Sealed hierarchies with pattern matching give the same operations-in-one-

© TechTribers. For personal use only. Do not redistribute.

place style with less ceremony, which is why Visitor is less common in modern Java. It remains standard in compilers, AST processing and libraries such as ANTLR and `java.nio.file.FileVisitor`.

Java

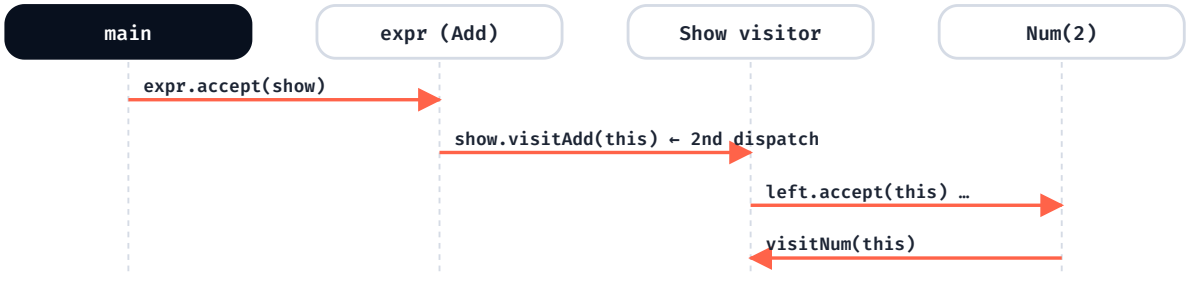
```
interface Node { <R> R accept(Visitor<R> v); }
interface Visitor<R> { R visitNum(Num n); R visitAdd(Add a); }
record Num(int v) implements Node { public <R> R accept(Visitor<R> vis) { return vis.visitNum(this);
} }
record Add(Node l, Node r) implements Node { public <R> R accept(Visitor<R> vis) { return
vis.visitAdd(this); } }

class Eval implements Visitor<Integer> {
    public Integer visitNum(Num n) { return n.v(); }
    public Integer visitAdd(Add a) { return a.l().accept(this) + a.r().accept(this); }
}
class Show implements Visitor<String> {
    public String visitNum(Num n) { return "" + n.v(); }
    public String visitAdd(Add a) { return "(" + a.l().accept(this) + " + " + a.r().accept(this) +
")"; }
}
Node expr = new Add(new Num(2), new Add(new Num(3), new Num(4)));
System.out.println(expr.accept(new Show()) + " = " + expr.accept(new Eval()));

// Modern alternative (Java 21): sealed types + pattern-matching switch, with compiler-checked
exhaustiveness
sealed interface Expr permits N, Plus { }
record N(int v) implements Expr { }
record Plus(Expr l, Expr r) implements Expr { }
static int eval(Expr e) { return switch (e) { case N n -> n.v(); case Plus p -> eval(p.l()) +
eval(p.r()); }; }
System.out.println(eval(new Plus(new N(2), new Plus(new N(3), new N(4)))));
```

▶ Output

```
(2 + (3 + 4)) = 9
9
```



Two virtual calls pick the method by BOTH types (node and visitor): Java's way to fake double dispatch.

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Say the one-line summary: "Visitor puts the operation outside the class hierarchy and uses a second virtual call to recover the concrete type."

Q470 OP-043 • Intermediate

What is the Null Object pattern?

> ⚡ **Quick answer**
> Instead of returning or storing `null` for "nothing", provide a real object that implements the same interface and does nothing (or returns safe defaults). Callers then use it like any other object, so they need no `null` checks, and a whole category of `NullPointerException` disappears.

JDK examples: `Collections.emptyList()`, `Optional.empty()`, `InputStream.nullInputStream()`, `Logger.getGlobal()` with disabled levels. Use it when "do nothing" is a valid behaviour. If absence is a real business case the caller must handle, return `Optional` or throw instead of hiding it.

Java

```
interface Logger { void log(String msg); }
class ConsoleLogger implements Logger { public void log(String m) { System.out.println("LOG: " + m);
} }
enum NoopLogger implements Logger { INSTANCE; public void log(String m) { } }        // the null
object: safe and stateless

class Service {
    private final Logger logger;
    Service(Logger logger) { this.logger = logger == null ? NoopLogger.INSTANCE : logger; }
    void run() { logger.log("started"); System.out.println("running"); }              // no null
checks anywhere
}
new Service(new ConsoleLogger()).run();
new Service(null).run();
```

▶ Output

```
LOG: started
running
running
```

> ⚠️ **Interview trap**
> Returning an empty collection instead of `null` is the most common null-object usage. Returning `null` for a collection or array is almost always a bug waiting for a NullPointerException.

© TechTribers. For personal use only. Do not redistribute.

Q471 OP-044

• Advanced

How do you design a class for inheritance, and when should a class be final?

⚡ Quick answer

Either **forbid** inheritance (`final` , sealed, or private constructors) or **design for it deliberately**: keep the algorithm's skeleton `final` (template method), expose a few documented `protected` hooks, never call overridable methods from constructors, and document self-use. Joshua Bloch: "Design and document for inheritance or else prohibit it."

Make a class `final` when: it's a value type (`String` , `Money`), its invariants depend on exact behaviour, or it was not written to be extended. **Allow extension when**: it's a framework base class with documented hooks. **Sealed types** (Java 17) are the middle path: the hierarchy is open to *chosen* subclasses only.

Checklist for an extensible class: a stable contract documented in Javadoc (`@implSpec`), few overridable methods, no overridable calls in constructors, and `equals` / `hashCode` decisions made explicitly.

Java

```
abstract class Pipeline {
    public final String run(String in) { return after(transform(before(in))); }      // skeleton is
final: subclasses can't break the order
    protected String before(String in) { return in.strip(); }                          // optional
hook with a sensible default
    protected abstract String transform(String in);                                    // required
step
    protected String after(String in) { return in; }
}
class Shout extends Pipeline {
    protected String transform(String in) { return in.toUpperCase(); }
}
System.out.println("[" + new Shout().run("  hello ") + "]");
```

▶ Output

```
[HELLO]
```

💡 Pro tip

Default to `final` for new classes and open them up when there's a real need. Removing `final` later is a compatible change; adding it later breaks existing subclasses.

© TechTribers. For personal use only. Do not redistribute.

Section 06

Low-Level Design: Object-Oriented Problems

Eight classic machine-coding rounds, each with a class diagram, the patterns used and a complete runnable solution.

Questions 472–479 · 8 questions



Q472 OP-045

• Advanced

Design a parking lot.

⚡ Quick answer

Model **Vehicle** (data), **Spot** (knows what it fits and whether it's free), **Ticket** (entry record), **ParkingLot** (finds a spot, issues tickets, charges on exit) and a **FeePolicy** interface (Strategy) so pricing can change without touching the lot. Start with requirements, then entities, then behaviour.

Discussion points interviewers expect: multiple levels (add a `Level` that owns spots), closest-spot allocation (a strategy for choosing spots), concurrency (synchronise `enter` / `exit` or use per-spot locks), payment, entry/exit gates, and extending `VehicleType` (an electric spot that fits CAR and charges).

Patterns used: **Strategy** (FeePolicy), **Factory** (could create spots per layout), **Singleton** is *not* needed: pass the lot where required.

Java

```
enum VehicleType { BIKE, CAR, TRUCK }
record Vehicle(String plate, VehicleType type) { }

class Spot {
    final int id; private final VehicleType fits; private Vehicle parked;
    Spot(int id, VehicleType fits) { this.id = id; this.fits = fits; }
    boolean canFit(Vehicle v) { return parked == null && v.type() == fits; }
    void park(Vehicle v) { parked = v; }
    void leave() { parked = null; }
}
record Ticket(String id, Vehicle vehicle, Spot spot, long inMinute) { }

interface FeePolicy { long fee(VehicleType type, long minutes); }                    // Strategy
class HourlyFee implements FeePolicy {
    public long fee(VehicleType type, long minutes) {
        long hours = Math.max(1, (minutes + 59) / 60);                               // round
up, minimum 1 hour
        long rate = switch (type) { case BIKE -> 10; case CAR -> 30; case TRUCK -> 60; };
        return hours * rate;
    }
}
```

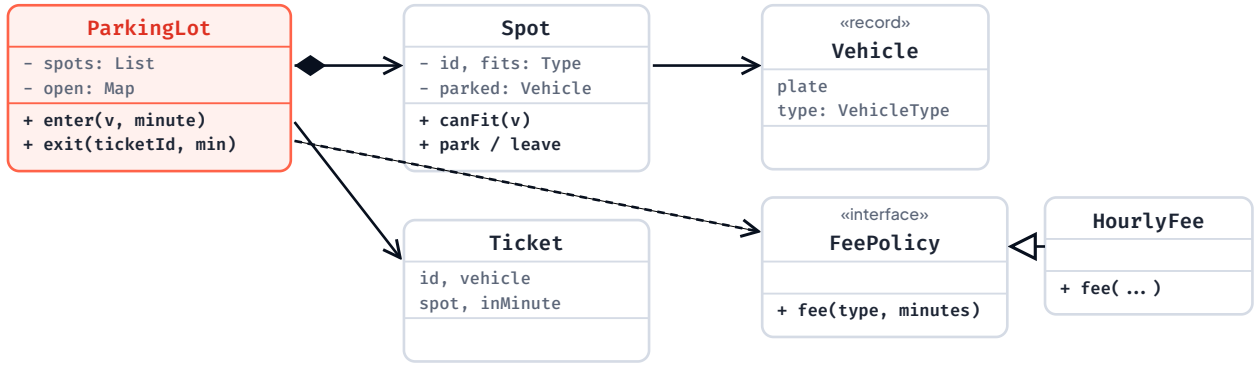

© TechTribers. For personal use only. Do not redistribute.

```
class ParkingLot {
    private final List<Spot> spots; private final FeePolicy policy;
    private final Map<String, Ticket> open = new HashMap<>(); private int seq;
    ParkingLot(List<Spot> spots, FeePolicy policy) { this.spots = spots; this.policy = policy; }

    Ticket enter(Vehicle v, long minute) {
        Spot s = spots.stream().filter(sp -> sp.canFit(v)).findFirst()
                .orElseThrow(() -> new IllegalStateException("lot full for " + v.type()));
        s.park(v);
        Ticket t = new Ticket("T" + (++seq), v, s, minute);
        open.put(t.id(), t);
        return t;
    }
    long exit(String ticketId, long minute) {
        Ticket t = open.remove(ticketId);
        t.spot().leave();
        return policy.fee(t.vehicle().type(), minute - t.inMinute());
    }
}
ParkingLot lot = new ParkingLot(List.of(new Spot(1, VehicleType.CAR), new Spot(2, VehicleType.CAR),
new Spot(3, VehicleType.BIKE)), new HourlyFee());
Ticket t1 = lot.enter(new Vehicle("MH12AB1234", VehicleType.CAR), 0);
lot.enter(new Vehicle("MH14CD5678", VehicleType.CAR), 10);
try { lot.enter(new Vehicle("MH01EF0001", VehicleType.CAR), 20); } catch (IllegalStateException e) {
System.out.println(e.getMessage()); }
System.out.println("fee for " + t1.id() + ": " + lot.exit(t1.id(), 125));
System.out.println("freed spot reused: " + lot.enter(new Vehicle("MH01EF0001", VehicleType.CAR),
130).spot().id);
```

▶ Output

```
lot full for CAR
fee for T1: 90
freed spot reused: 1
```



Strategy: swap FeePolicy (hourly, flat, peak-time) without touching ParkingLot. Spot decides what fits.

💡 **Pro tip**

State assumptions first (one lot, minute-based time, spots typed by vehicle). Interviewers grade the clarity of the model more than the amount of code.

© TechTribers. For personal use only. Do not redistribute.

Q473 OP-046

• Advanced

Design a vending machine using the State pattern.

⚡ Quick answer

Represent each situation (**Idle**, **HasMoney**, later **Dispensing**, **OutOfService**) as a **State** object. The machine delegates `insert` and `select` to its current state, and each state decides what happens and which state comes next. This replaces a tangle of `if (state == ...)` checks.

Adding an `OutOfService` state means adding one class; existing states don't change (Open/Closed). The alternative, an `enum State` with a `switch` in every method, works for small machines but grows a branch per state in every operation. Real extensions: coin denominations and change-making (a Chain of Responsibility), refund, admin restock, and persistence of stock.

Java

```
interface State { String insert(Machine m, int coin); String select(Machine m, String item); }

class Machine {
    private final Map<String, Integer> prices = Map.of("COLA", 40, "CHIPS", 25);
    private final Map<String, Integer> stock = new HashMap<>(Map.of("COLA", 1, "CHIPS", 5));
    private State state = new Idle();
    int balance;
    void setState(State s) { state = s; }
    String insert(int coin) { return state.insert(this, coin); }
    String select(String item) { return state.select(this, item); }
    int price(String item) { return prices.get(item); }
    boolean inStock(String item) { return stock.getOrDefault(item, 0) > 0; }
    void take(String item) { stock.merge(item, -1, Integer::sum); }
}
class Idle implements State {
    public String insert(Machine m, int coin) { m.balance += coin; m.setState(new HasMoney());
return "balance " + m.balance; }
    public String select(Machine m, String item) { return "insert coin first"; }
}
class HasMoney implements State {
    public String insert(Machine m, int coin) { m.balance += coin; return "balance " + m.balance; }
    public String select(Machine m, String item) {
        if (!m.inStock(item)) return item + " sold out";
        if (m.balance < m.price(item)) return "need " + (m.price(item) - m.balance) + " more";
        m.take(item);
        int change = m.balance - m.price(item);
        m.balance = 0;
        m.setState(new Idle());                                  // the state object decides the
next state
        return "dispensed " + item + ", change " + change;
    }
}
Machine m = new Machine();
System.out.println(m.select("COLA"));
System.out.println(m.insert(20));
System.out.println(m.select("COLA"));
System.out.println(m.insert(50));
System.out.println(m.select("COLA"));
```

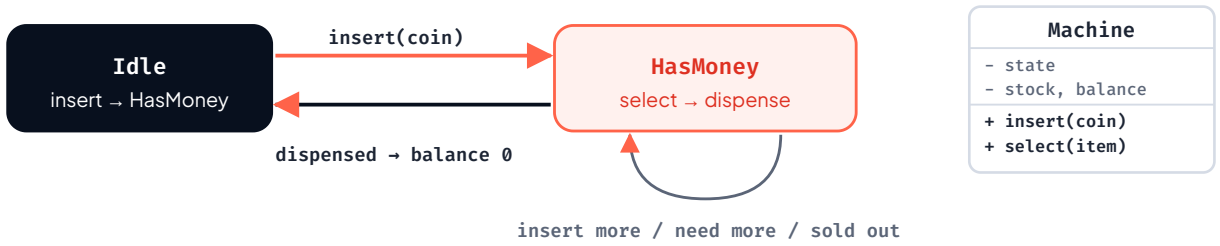

© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(m.insert(50));
System.out.println(m.select("COLA"));
```

▶ Output

```
insert coin first
balance 20
need 20 more
balance 70
dispensed COLA, change 30
balance 50
COLA sold out
```

State pattern: behaviour changes with the state object, not with if/else on flags



Machine delegates insert/select to its current State; each State returns the next one.

⚠️ **Interview trap**

Don't forget to reset `balance` and move to `Idle` after dispensing. State machines fail most often on the transitions nobody drew.

Q474 OP-047 • Advanced

Design an ATM (cash dispensing and transaction flow).

⚡ **Quick answer**

Use a **Chain of Responsibility** of dispensers (₹2000 → ₹500 → ₹100): each takes as many notes as it can and passes the remainder on. Keep the flow safe: validate the PIN and amount, **compute** the notes first, and **commit** (debit the account) last so a failure never leaves inconsistent state.

Extensions to discuss: ATM **states** (idle, card inserted, authenticated, out of cash) using the State pattern from the previous question; per-note cash inventory (a dispenser must stop when it runs out, falling through to the next); daily limits; audit log; and a `Bank` interface so the ATM depends on an abstraction (DIP).

Concurrency: two ATMs debiting the same account need an atomic debit (database transaction or `compareAndSet`), not just a check-then-act.

Java

```
class Dispenser {
    private Dispenser next; private final int note;
```


© TechTribers. For personal use only. Do not redistribute.

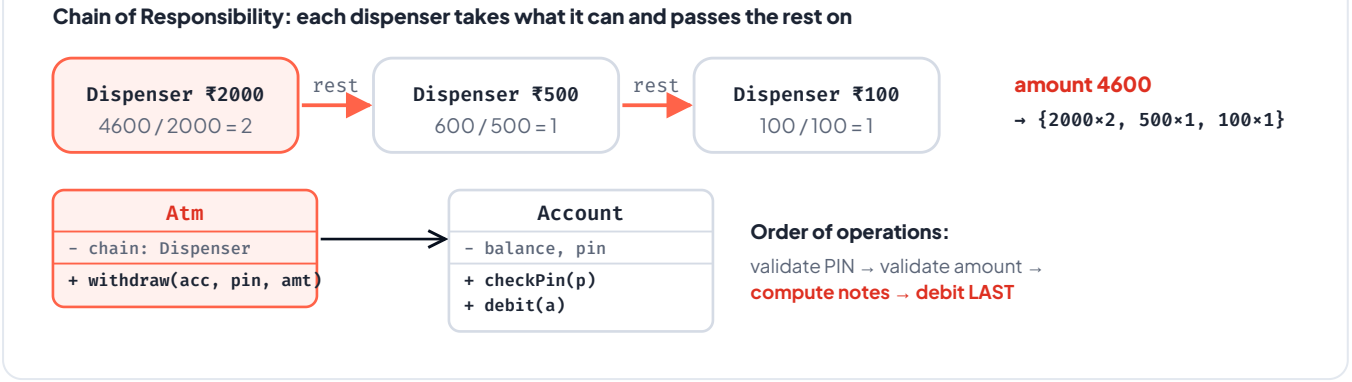
```
    Dispenser(int note) { this.note = note; }
    Dispenser then(Dispenser n) { next = n; return n; }                    // build the
chain fluently
    void dispense(int amount, Map<Integer, Integer> out) {
        int count = amount / note;
        if (count > 0) out.put(note, count);
        int rest = amount % note;
        if (rest > 0) {
            if (next == null) throw new IllegalArgumentException("cannot dispense " + amount);
            next.dispense(rest, out);                                      // pass the
remainder on
        }
    }
}
class Account {
    private int balance; private final String pin;
    Account(int balance, String pin) { this.balance = balance; this.pin = pin; }
    boolean checkPin(String p) { return pin.equals(p); }
    void debit(int a) { if (a > balance) throw new IllegalStateException("insufficient balance");
balance -= a; }
    int balance() { return balance; }
}
class Atm {
    private final Dispenser chain;
    Atm(Dispenser chain) { this.chain = chain; }
    Map<Integer, Integer> withdraw(Account acc, String pin, int amount) {
        if (!acc.checkPin(pin)) throw new SecurityException("wrong PIN");
        if (amount % 100 != 0) throw new IllegalArgumentException("amount must be a multiple of
100");
        Map<Integer, Integer> notes = new TreeMap<>(Comparator.reverseOrder());
        chain.dispense(amount, notes);                                     // compute
first (no side effects)
        acc.debit(amount);                                                 // commit
last
        return notes;
    }
}
Dispenser d2000 = new Dispenser(2000);
d2000.then(new Dispenser(500)).then(new Dispenser(100));
Atm atm = new Atm(d2000);
Account acc = new Account(10_000, "1234");

System.out.println(atm.withdraw(acc, "1234", 4600) + " balance=" + acc.balance());
for (Runnable attempt : List.<Runnable>of(() -> atm.withdraw(acc, "0000", 100), () ->
atm.withdraw(acc, "1234", 4650), () -> atm.withdraw(acc, "1234", 9000))) {
    try { attempt.run(); } catch (RuntimeException e) { System.out.println(e.getMessage()); }
}
System.out.println("balance still " + acc.balance());
```

▶ Output

```
{2000=2, 500=1, 100=1} balance=5400
wrong PIN
amount must be a multiple of 100
insufficient balance
balance still 5400
```


© TechTribers. For personal use only. Do not redistribute.



⚠️ **Interview trap**

Debiting before dispensing, or dispensing before validating, leaves the account and the cash tray inconsistent when something fails. Order of operations is the real design problem here.

Q475 OP-048 • Advanced

Design a library management system.

⚡ **Quick answer**

Core model: **Book** and **Loan** as immutable records, **Patron** holding current loans, and a **Library** that owns the catalogue, stock counts and the rules (loan limit, loan period, fine per day). Keep rules as named constants or a policy object so they can change without rewriting the flow.

Natural extensions: **reservations** (a queue per ISBN, `Observer` notifies the next patron on return), search by title/author (an index), book copies vs titles (a `BookCopy` with its own id and condition), membership tiers with different limits (a `LoanPolicy` strategy), and persistence through a `Repository` interface.

```java
record Book(String isbn, String title) { }
record Loan(Book book, int dayBorrowed) { }
class Patron {
    final String name;
    final List<Loan> loans = new ArrayList<>();
    Patron(String name) { this.name = name; }
}
class Library {
    static final int MAX_LOANS = 2, LOAN_DAYS = 14, FINE_PER_DAY = 2;
    private final Map<String, Book> catalog = new HashMap<>();
    private final Map<String, Integer> copies = new HashMap<>();

    void addBook(Book b, int n) { catalog.put(b.isbn(), b); copies.merge(b.isbn(), n, Integer::sum);
}

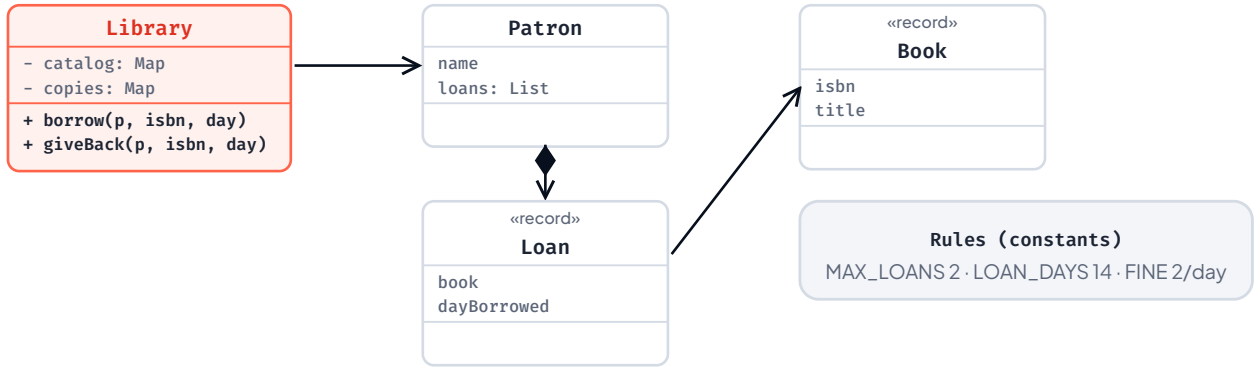
    void borrow(Patron p, String isbn, int day) {
        if (p.loans.size() >= MAX_LOANS) throw new IllegalStateException(p.name + " reached the
limit of " + MAX_LOANS);
```


© TechTribers. For personal use only. Do not redistribute.

```
        if (copies.getOrDefault(isbn, 0) == 0) throw new IllegalStateException("no copies left of "
+ catalog.get(isbn).title());
        copies.merge(isbn, -1, Integer::sum);
        p.loans.add(new Loan(catalog.get(isbn), day));
    }
    int giveBack(Patron p, String isbn, int day) {                    // returns the fine
        Loan loan = p.loans.stream().filter(l ->
l.book().isbn().equals(isbn)).findFirst().orElseThrow();
        p.loans.remove(loan);
        copies.merge(isbn, 1, Integer::sum);
        return Math.max(0, day - loan.dayBorrowed() - LOAN_DAYS) * FINE_PER_DAY;
    }
}
Library lib = new Library();
lib.addBook(new Book("111", "Effective Java"), 1);
lib.addBook(new Book("222", "Clean Code"), 2);
Patron asha = new Patron("Asha"), ravi = new Patron("Ravi");
lib.borrow(asha, "111", 1);
lib.borrow(asha, "222", 1);
try { lib.borrow(asha, "222", 2); } catch (IllegalStateException e) {
System.out.println(e.getMessage()); }
try { lib.borrow(ravi, "111", 2); } catch (IllegalStateException e) {
System.out.println(e.getMessage()); }
System.out.println("fine: " + lib.giveBack(asha, "111", 20));
lib.borrow(ravi, "111", 21);
System.out.println(ravi.name + " has " + ravi.loans.size() + " loan(s)");
```

▶ Output

```
Asha reached the limit of 2
no copies left of Effective Java
fine: 10
Ravi has 1 loan(s)
```



Fine = max(0, returnDay – borrowDay – 14) × 2. Rules live in Library, data lives in records.

💡 Pro tip

Separate *titles* (`Book`) from *physical copies*. Many weak designs mix them, and then can't answer "which copy does Asha hold?".

© TechTribers. For personal use only. Do not redistribute.

Q476 OP-049

• Intermediate

Design a tic-tac-toe game that scales to n × n.

> ⚡ **Quick answer**
>
> Split responsibilities: **Board** owns geometry (placing marks, detecting a win, being full), **Player** is plain data (name + symbol), and **Game** owns turns and the rules of play. Making the board size a constructor argument gives n × n with no other change.

Optimisation question: checking all rows/columns each move is O(n²). Keep **row, column and diagonal counters per player** and update them on each move, so win detection is O(1) per move.

Extensions: undo (a `Command` per move), an AI player behind a `Player` interface (`nextMove(board)`), more than two players, and a UI that observes the game through `Observer`.

Java

```
class Board {
    private final char[][] cells; private final int n; private int moves;
    Board(int n) { this.n = n; cells = new char[n][n]; for (char[] row : cells) Arrays.fill(row,
'.'); }
    boolean place(int r, int c, char sym) {
        if (r < 0 || c < 0 || r >= n || c >= n || cells[r][c] != '.') return false;
        cells[r][c] = sym; moves++; return true;
    }
    boolean wins(char s) {
        boolean diag = true, anti = true;
        for (int i = 0; i < n; i++) {
            boolean row = true, col = true;
            for (int j = 0; j < n; j++) { row &= cells[i][j] == s; col &= cells[j][i] == s; }
            if (row || col) return true;
            diag &= cells[i][i] == s; anti &= cells[i][n - 1 - i] == s;
        }
        return diag || anti;
    }
    boolean full() { return moves == n * n; }
    @Override public String toString() { StringBuilder sb = new StringBuilder(); for (char[] row :
cells) sb.append(row).append('\n'); return sb.toString(); }
}
record Player(String name, char symbol) { }
class Game {
    private final Board board; private final Player[] players; private int turn;
    Game(int n, Player a, Player b) { board = new Board(n); players = new Player[]{a, b}; }
    String play(int r, int c) {
        Player p = players[turn % 2];
        if (!board.place(r, c, p.symbol())) return "invalid move, " + p.name() + " try again";
        if (board.wins(p.symbol())) return p.name() + " wins";
        if (board.full()) return "draw";
        turn++;
        return "next: " + players[turn % 2].name();
    }
    Board board() { return board; }
}
Game g = new Game(3, new Player("Asha", 'X'), new Player("Ravi", 'O'));
int[][] moves = {{0, 0}, {1, 1}, {0, 1}, {1, 1}, {2, 2}, {0, 2}};
```


© TechTribers. For personal use only. Do not redistribute.

```
for (int[] mv : moves) System.out.println(g.play(mv[0], mv[1]));
System.out.print(g.board());
```

▶ Output

```
next: Ravi
next: Asha
next: Ravi
invalid move, Ravi try again
next: Asha
Asha wins
XXX
.O.
..O
```



Board knows geometry, Game knows turns and rules, Player is plain data. n×n works by changing one constructor argument.

💡 **Pro tip**

Interviewers love the O(1) counters follow-up. Mention it before they ask.

Q477 OP-050 • Advanced

Design an elevator system.

⚡ **Quick answer**

Start with one elevator using the **SCAN** idea: keep moving in the current direction serving requested floors in order, then reverse. A sorted set of pending stops (`TreeSet`) makes "next stop above/below me" a cheap `ceiling` / `floor` lookup. Then generalise to several elevators with a dispatching strategy.

Scaling to a building: an `ElevatorController` receives hall calls (floor + direction) and picks an elevator via a **dispatch Strategy** (nearest idle elevator, or the one already moving toward the call). Each elevator can use the **State** pattern (idle, moving up, moving down, doors open, out of service).

Concurrency: requests arrive while the elevator moves, so `request` must be thread-safe (a concurrent sorted set or a single-threaded event loop per elevator).

© TechTribers. For personal use only. Do not redistribute.

Java

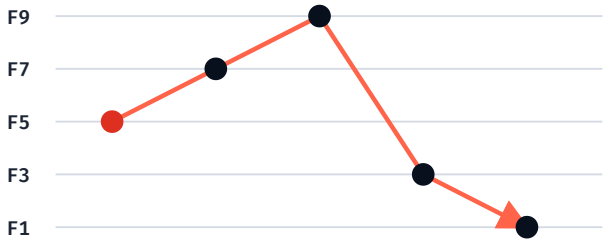
```java
enum Direction { UP, DOWN, IDLE }
class Elevator {
    private int floor; private Direction dir = Direction.IDLE;
    private final TreeSet<Integer> stops = new TreeSet<>();
    Elevator(int startFloor) { floor = startFloor; }
    void request(int f) { if (f != floor) stops.add(f); }

    List<Integer> run() {                                          // returns floors in the order they are served
        List<Integer> served = new ArrayList<>();
        while (!stops.isEmpty()) {
            Integer next = dir == Direction.DOWN ? stops.floor(floor) : stops.ceiling(floor);   // nearest stop in this direction
            if (next == null) { dir = dir == Direction.UP ? Direction.DOWN : Direction.UP; continue; }   // nothing ahead: reverse
            dir = next > floor ? Direction.UP : Direction.DOWN;
            floor = next;
            stops.remove(next);
            served.add(next);
        }
        dir = Direction.IDLE;
        return served;
    }
}
Elevator e = new Elevator(5);
for (int f : new int[]{7, 3, 9, 1}) e.request(f);
System.out.println("served: " + e.run());
```

▶ Output

```
served: [7, 9, 3, 1]
```

SCAN (elevator) strategy from floor 5, requests {7, 3, 9, 1}: serve in the current direction, then reverse



served: [7, 9, 3, 1]

Elevator
- floor, dir: Direction - stops: TreeSet
+ request(floor) + run(): List

TreeSet gives **ceiling() / floor()** in O(log n)

⚠ **Interview trap**

A FIFO queue of requests is the naive answer and zig-zags between floors. SCAN-style ordering (serve in the current direction first) is what interviewers want to hear.

© TechTribers. For personal use only. Do not redistribute.

🛒 **Q478** OP-051 • Intermediate

Design a shopping cart with flexible pricing rules (discounts, coupons, tax).

⚡ **Quick answer**

Treat each rule as a **PricingRule** (Strategy) that takes the running amount and returns a new one, and let the cart apply the rules in order. New promotions are new rule objects (or lambdas), and `Cart.total()` never changes, which is Open/Closed in practice. Rule order is a business decision, so make it explicit.

Design notes: use `BigDecimal` for money in real systems (doubles are used here for readability); give each rule a name and priority so a coupon can't be applied twice; and keep tax last or first depending on the jurisdiction (tax-inclusive vs exclusive pricing).

Patterns: **Strategy** for rules, **Chain/Pipeline** for ordering, **Builder** for the fluent `Cart`, and **Decorator** if rules should wrap each other (a coupon wrapping a base price).

Java

```
record Item(String sku, double price, int qty) { double total() { return price * qty; } }

interface PricingRule { double apply(double amount, List<Item> items); }

class Cart {
    private final List<Item> items = new ArrayList<>();
    private final List<PricingRule> rules = new ArrayList<>();
    Cart add(Item i) { items.add(i); return this; }
    Cart rule(PricingRule r) { rules.add(r); return this; }
    double total() {
        double amount = items.stream().mapToDouble(Item::total).sum();
        for (PricingRule r : rules) amount = r.apply(amount, items);                // pipeline:
output of one rule feeds the next
        return amount;
    }
}
PricingRule tenPercentOver1000 = (amount, items) -> amount > 1000 ? amount * 0.9 : amount;
PricingRule gst18 = (amount, items) -> amount * 1.18;
PricingRule bulkBonus = (amount, items) -> items.stream().anyMatch(i -> i.qty() >= 5) ? amount - 25
: amount;

Cart cart = new Cart().add(new Item("A", 500, 2)).add(new Item("B", 200, 3));
System.out.printf("%.2f%n", cart.rule(tenPercentOver1000).rule(gst18).total());          // 1600 ->
1440 -> 1699.20

// Percentage rules commute, but a fixed amount does not: the order of rules is a business decision
Cart bulk1 = new Cart().add(new Item("C", 100, 6)).rule(bulkBonus).rule(gst18);          // (600 -
25) * 1.18
Cart bulk2 = new Cart().add(new Item("C", 100, 6)).rule(gst18).rule(bulkBonus);          // 600 *
1.18 - 25
System.out.printf("%.2f %.2f%n", bulk1.total(), bulk2.total());
```

▶ **Output**

```
1699.20
678.50 683.00
```


© TechTribers. For personal use only. Do not redistribute.

Pipeline of pricing rules: each rule is a strategy; order matters



Percentage rules commute, fixed amounts do not: (600 − 25) × 1.18 = 678.50 but 600 × 1.18 − 25 = 683.00.

New rule (coupon, loyalty, free delivery) = new PricingRule lambda; Cart.total() never changes (Open/Closed).

⚠️ **Interview trap**

Money as `double` loses precision (`0.1 + 0.2`). Use `BigDecimal` or integer paise, and fix a rounding mode, before anything touches real prices.

Q479 OP-052 • Advanced

Design a notification system (email, SMS, push) with topics and subscribers.

⚡ **Quick answer**

Combine three patterns: **Observer** (topics with subscribers), **Strategy** (a `NotificationChannel` interface with email/SMS/push implementations) and **Factory** (create a channel by name). Publishers know only the service, the service knows only the interface, and new channels need no change to existing code.

Production concerns to raise: **delivery guarantees** and retries (a failing channel must not block the others; send asynchronously via a queue or Kafka), **user preferences** (opt-out per channel), **templates**, rate limiting per user, and idempotency so a retry doesn't send twice.

If publishing and sending must be decoupled for scale, the in-memory `publish` becomes a message broker, and each channel a consumer group.

Java

```
interface NotificationChannel { void send(String user, String msg); }                // Strategy
class EmailChannel implements NotificationChannel { public void send(String u, String m) {
System.out.println("EMAIL to " + u + ": " + m); } }
class SmsChannel implements NotificationChannel { public void send(String u, String m) {
System.out.println("SMS to " + u + ": " + m); } }
class PushChannel implements NotificationChannel { public void send(String u, String m) {
System.out.println("PUSH to " + u + ": " + m); } }

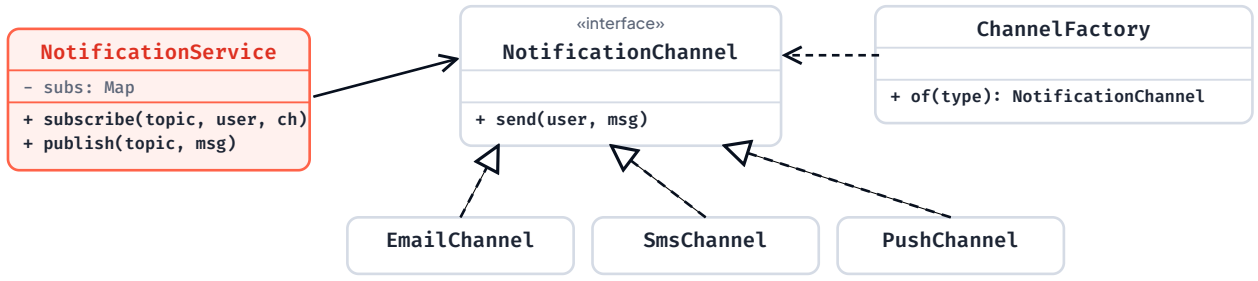
class ChannelFactory {                                                                // Factory
    static NotificationChannel of(String type) {
        return switch (type) {
            case "email" -> new EmailChannel();
            case "sms" -> new SmsChannel();
            case "push" -> new PushChannel();
            default -> throw new IllegalArgumentException("unknown channel " + type);
        };
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
}
class NotificationService {                                                    //
Observer: topic -> subscribers
    private record Subscription(String user, NotificationChannel channel) { }
    private final Map<String, List<Subscription>> subs = new HashMap<>();
    void subscribe(String topic, String user, NotificationChannel ch) {
        subs.computeIfAbsent(topic, t -> new ArrayList<>()).add(new Subscription(user, ch));
    }
    void publish(String topic, String msg) {
        for (Subscription s : subs.getOrDefault(topic, List.of())) s.channel().send(s.user(), "[" +
topic + "] " + msg);
    }
}
NotificationService svc = new NotificationService();
svc.subscribe("orders", "asha", ChannelFactory.of("email"));
svc.subscribe("orders", "asha", ChannelFactory.of("push"));
svc.subscribe("orders", "ravi", ChannelFactory.of("sms"));
svc.subscribe("offers", "ravi", ChannelFactory.of("email"));
svc.publish("orders", "order 42 shipped");
svc.publish("offers", "20% off today");
svc.publish("nobody", "no subscribers, nothing happens");
try { ChannelFactory.of("fax"); } catch (IllegalArgumentException e) {
System.out.println(e.getMessage()); }
```

▶ Output

```
EMAIL to asha: [orders] order 42 shipped
PUSH to asha: [orders] order 42 shipped
SMS to ravi: [orders] order 42 shipped
EMAIL to ravi: [offers] 20% off today
unknown channel fax
```



Observer (topics → subscribers) + Strategy (channels) + Factory (create by name): each part changes independently.

💡 Pro tip

Say out loud which pattern solves which problem. "Observer decouples publishers from subscribers, Strategy decouples delivery from the channel, Factory decouples creation." That's what the question is testing.

© TechTribers. For personal use only. Do not redistribute.

Series progress

479 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 12 starts at Q480.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.